- ViandasSemana
- ViandasSemana — Visión general
  - 1. El problema
  - 2. La solución
  - 3. Actores
  - 4. Módulos
  - 5. Reglas de negocio
  - 6. Mapa de la documentación
- Diagramas de flujo de datos (DFD)
  - Nivel 0 — Diagrama de contexto (consigna 4.1)
  - Nivel 1 (consigna 4.2)
  - Nivel 2 (consigna 4.3)
- Casos de uso (consigna 4.4)
  - 1. Diagrama general
  - 2. Actores
  - 3. Catálogo por módulo
  - 4. Relaciones `<<include>>` y `<<extend>>`
  - 5. Narrativas de los tres casos de uso principales
- Modelo de datos — DER (consigna 4.5)
  - Cardinalidades
  - Diccionario de datos
  - Consultas para mostrar en vivo
- Arquitectura y despliegue (consigna 4.6)
  - 1. Componentes desplegables
  - 2. Comunicaciones
  - 3. Secretos y su recorrido
  - 4. Del push a producción
  - 5. Ambiente local vs. productivo
- Decisiones técnicas (consigna 4.7)
  - Declaración de uso de inteligencia artificial (consigna 8)
- Módulo 1 — Acceso
  - 1. Propósito
  - 2. Dónde está en el código
  - 3. Casos de uso
  - 4. DFD Nivel 2 — Proceso 1 “Gestionar acceso”
  - 5. Datos que usa
  - 6. API
  - 7. Reglas y validaciones (dónde está cada una)
  - 8. Pantallas
  - 9. Tests que lo cubren
  - 10. Para modificarlo
- Módulo 2 — Menús
  - 1. Propósito
  - 2. Dónde está en el código
  - 3. Casos de uso
  - 4. DFD Nivel 2 — Proceso 2 “Gestionar menús”
  - 5. Datos que usa
  - 6. API
  - 7. Reglas y validaciones
  - 8. Pantallas
  - 9. Tests que lo cubren (`menus.test.ts`)

    - 10. Para modificarlo
- Módulo 3 — Pedidos
    - 1. Propósito
    - 2. Dónde está en el código
    - 3. Casos de uso
    - 4. DFD Nivel 2 — Proceso 3 “Gestionar pedidos”
    - 5. Datos que usa
    - 6. API
    - 7. Reglas y validaciones
    - 8. Pantallas
    - 9. Tests que lo cubren
    - 10. Para modificarlo
- Módulo 4 — Pagos
    - 1. Propósito
    - 2. Dónde está en el código
    - 3. Casos de uso
    - 4. DFD Nivel 2 — Proceso 4 “Procesar pagos”
    - 5. Datos que usa
    - 6. API y servicios externos
    - 7. Reglas y validaciones
    - 8. Pantallas
    - 9. Tests que lo cubren (`pagos.test.ts`, 14)
    - 10. Para modificarlo
- Módulo 5 — Plataforma (transversal)
    - 1. Propósito
    - 2. Dónde está en el código
    - 3. Validación y errores
    - 4. Observabilidad (proceso 5 “Registrar eventos”)
    - 5. Salud y seguridad de la API
    - 6. Configuración
    - 7. Tests
    - 8. Pipeline de CI/CD
    - 9. Para modificarlo

ViandasSemana

Documentación del Trabajo Práctico Integrador

Segundo Enriquez Gatti · Roman Oberlander

6.° 1.ª — E.T. N.° 21 — 2026

ViandasSemana — Visión general

Integrantes: Segundo Enriquez Gatti · Roman Oberlander — 6.° 1.ª, E.T. N.° 21

1. El problema

Una persona que vende viandas caseras recibe los pedidos por mensajes. Cuando aumentan los clientes, los pedidos se acumulan, cuesta organizarlos y se pueden aceptar más viandas de las que se pueden cocinar en el día. Además, el cobro queda librado a la confianza: hay reservas que nunca se pagan.

2. La solución

ViandasSemana es un sistema web donde quien cocina publica el menú de cada día con un **cupo máximo**, un **horario de corte** y un **precio**, y los clientes reservan y pagan sus viandas con **Mercado Pago**. El sistema no deja pedir más de lo que hay, ni pedir o cancelar después del horario de corte, y sólo cuenta como reserva confirmada la que está pagada. Quien cocina ve en todo momento cuántas viandas tiene que preparar cada día.

3. Actores

Actor	Tipo	Qué hace
Visitante	humano	Crea su cuenta
Cliente	humano (rol `CLIENTE`)	Consulta menús, reserva y paga, consulta y cancela sus pedidos
Cocinero	humano (rol `COCINERO`)	Carga, modifica y borra menús; consulta las viandas a cocinar
Mercado Pago	sistema externo	Cobra, avisa los pagos (webhook) y hace devoluciones
Servicio de correo (SMTP de Gmail)	sistema externo	Entrega los códigos para recuperar la contraseña
Plataforma de logs (Render Logs)	sistema externo	Guarda los eventos para consultar qué pasó en producción

“Usuario” es la generalización de Cliente y Cocinero (lo que pueden hacer los dos: iniciar y cerrar sesión, recuperar la contraseña y consultar menús).

4. Módulos

El sistema está dividido en cinco módulos. Cada uno tiene su propio documento con casos de uso, plantillas, DFD de nivel 2, datos, endpoints, reglas, pantallas, tests y una guía de “dónde tocar”.

Módulo	Proceso del DFD	Casos de uso	Backend	Frontend	Tablas
1. Acceso	1 Gestionar acceso	CU-01 a CU-04	`routes/auth.ts`, `middleware/auth.ts`, `lib/mailer.ts`	`Acceso.tsx`, `Recuperar.tsx`	`Rol`, `Usuario`, `CodigoRecuperacion`

Módulo	Proceso del DFD	Casos de uso	Backend	Frontend	Tablas
2. Menús	2 Gestionar menús	CU-05 a CU-08	`routes/menus.ts`	`Menus.tsx`	`MenuDia`
3. Pedidos	3 Gestionar pedidos	CU-09 a CU-12	`routes/pedidos.ts`, `domain/reglasPedido.ts`	`Menus.tsx`, `Pedidos.tsx`	`Pedido`
4. Pagos	4 Procesar pagos	CU-13 a CU-17	`routes/pagos.ts`, `lib/pagos.ts`, `lib/mercadoPago.ts`	`App.tsx`, `Pedidos.tsx`	`Pago`
5. Plataforma	5 Registrar eventos	(transversal)	`app.ts`, `middleware/errorHandler.ts`, `lib/logger.ts`, `config/env.ts`	`api/cliente.ts`	—

5. Reglas de negocio

N.º	Regla	Dónde se controla
RN-1	No se puede pedir más viandas que el cupo disponible del día	`validarNuevoPedido` + bloqueo `FOR UPDATE` (módulo 3)
RN-2	No se puede pedir después del horario de corte (hora de Argentina)	`validarNuevoPedido` (módulo 3)
RN-3	No se puede cancelar después del horario de corte	`validarCancelacion` (módulo 3)
RN-4	Sólo el cocinero carga, modifica y borra menús y ve todos los pedidos; el cliente sólo ve y cancela los suyos	`exigirRol` y filtros por usuario (módulos 1 y 3)
RN-5	Una reserva se paga completa y sólo queda confirmada cuando Mercado Pago aprueba el pago por el monto exacto	`procesarPago` (módulo 4)
RN-6	Una reserva sin pagar ocupa cupo hasta 30 minutos (nunca después del corte); luego se cancela sola	`vencimientoDelPago`, `vencerReservasImpagas` (módulo 4)
RN-7	Cancelar una reserva pagada devuelve el dinero; un pago no esperado también se devuelve	`devolverPago` (módulos 3 y 4)
RN-8	Un menú por fecha; un menú con pedidos no se borra	`routes/menus.ts` (módulo 2)

6. Mapa de la documentación

Documento	Sección de la consigna
Visión general (este documento)	2 — Propuesta (actualizada)
Diagramas de flujo de datos	4.1, 4.2 y 4.3
Casos de uso	4.4
Modelo de datos (DER)	4.5

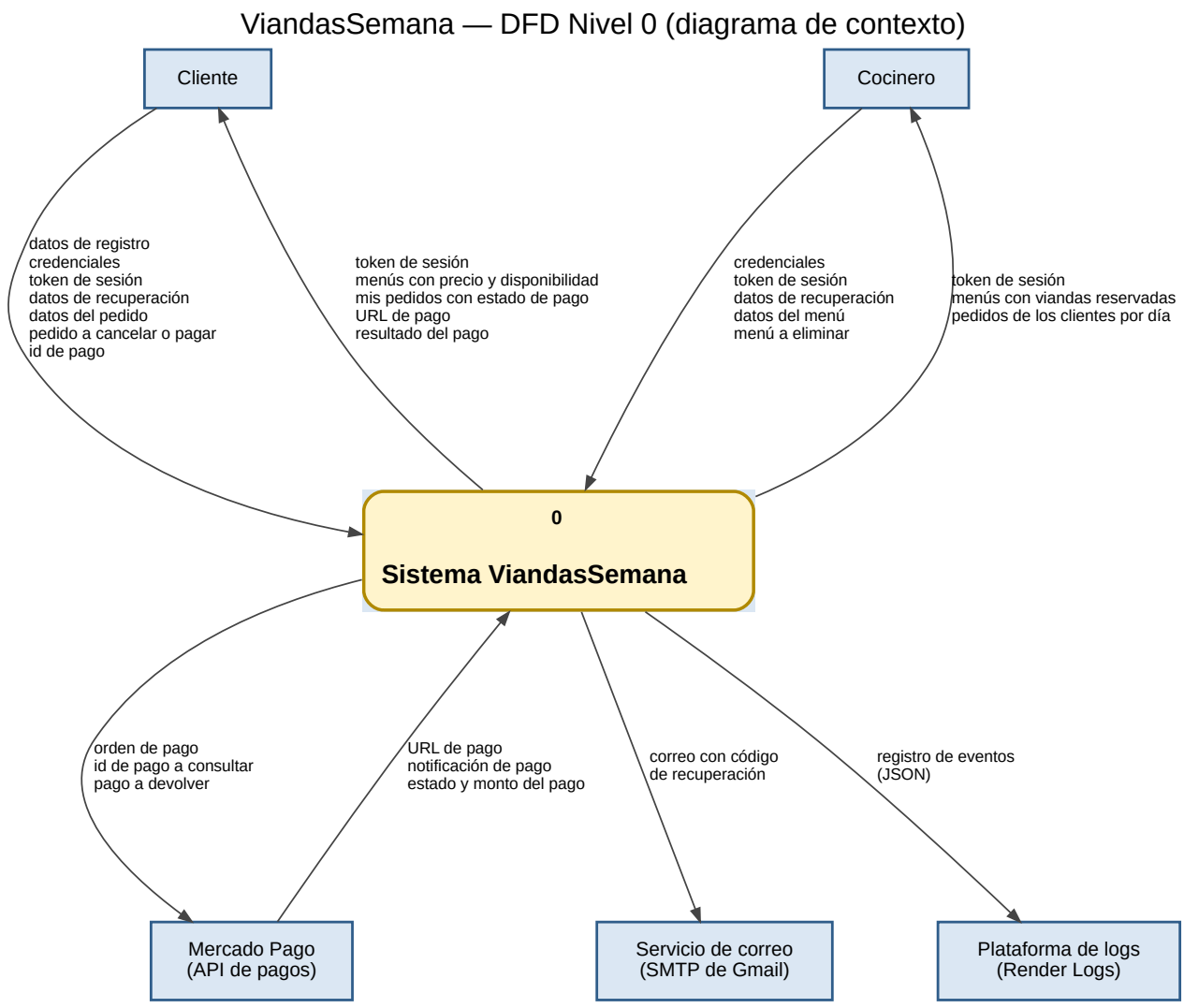
Documento	Sección de la consigna
Arquitectura y despliegue	4.6
Decisiones técnicas y uso de IA	4.7 y 8
Módulos 1 a 5	Detalle por módulo (plantillas de todos los casos de uso, DFD nivel 2, API, tests)

Diagramas de flujo de datos (DFD)

Notación **Gane-Sarson**: entidades externas en rectángulos azules, procesos en rectángulos redondeados amarillos con su número, almacenes de datos en rectángulos abiertos verdes con su código (D1…D5). Cada flujo lleva el **dato** que viaja, no la acción que lo origina. Los procesos de otro nivel que aparecen en un diagrama de nivel 2 se dibujan en gris.

Todas las peticiones de Cliente y Cocinero (salvo registro, inicio de sesión y recuperación) llevan el token de sesión; por eso “token de sesión” aparece como flujo de entrada en el Nivel 0 y llega al proceso 1, que entrega “identidad y rol” a los demás procesos.

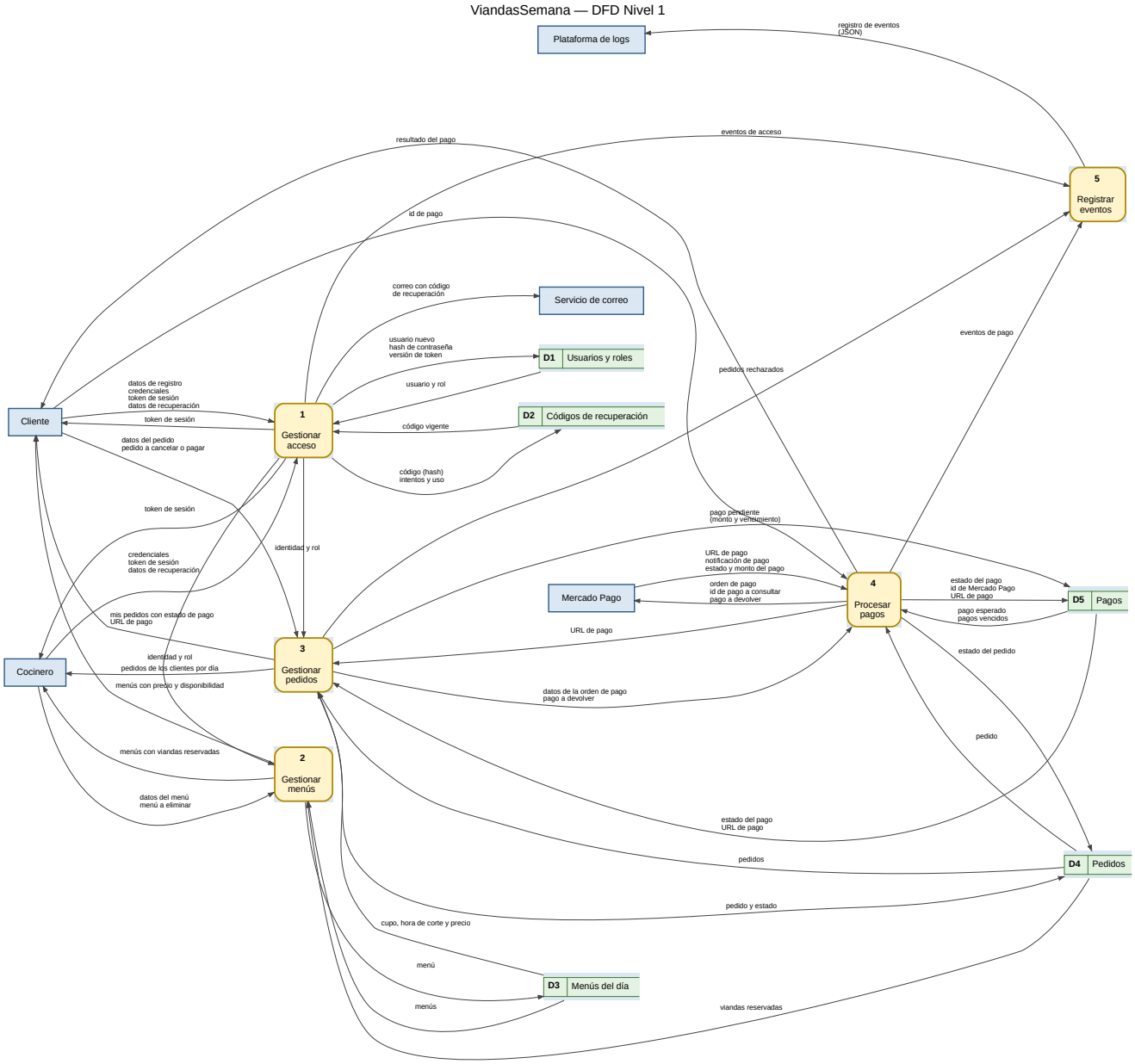
Nivel 0 — Diagrama de contexto (consigna 4.1)



DFD Nivel 0

Entidad externa	Flujos de entrada al sistema	Flujos de salida del sistema
Cliente	datos de registro · credenciales · token de sesión · datos de recuperación · datos del pedido · pedido a cancelar o pagar · id de pago	token de sesión · menús con precio y disponibilidad · mis pedidos con estado de pago · URL de pago · resultado del pago
Cocinero	credenciales · token de sesión · datos de recuperación · datos del menú · menú a eliminar	token de sesión · menús con viandas reservadas · pedidos de los clientes por día
Mercado Pago	URL de pago · notificación de pago · estado y monto del pago	orden de pago · id de pago a consultar · pago a devolver
Servicio de correo	—	correo con código de recuperación
Plataforma de logs	—	registro de eventos (JSON)

Nivel 1 (consigna 4.2)



DFD Nivel 1

N.º	Proceso	Módulo	Almacenes
1	Gestionar acceso	Acceso	D1 Usuarios y roles, D2 Códigos de recuperación
2	Gestionar menús	Menús	D3 Menús del día, D4 Pedidos (lectura)
3	Gestionar pedidos	Pedidos	D3, D4, D5
4	Procesar pagos	Pagos	D4, D5
5	Registrar eventos	Plataforma	— (los eventos van a la plataforma de logs)

Almacén	Tabla(s) de la base
D1 Usuarios y roles	`Usuario`, `Rol`

Almacén	Tabla(s) de la base
D2 Códigos de recuperación	`CodigoRecuperacion`
D3 Menús del día	`MenuDia`
D4 Pedidos	`Pedido`
D5 Pagos	`Pago`

Consistencia entre el Nivel 0 y el Nivel 1

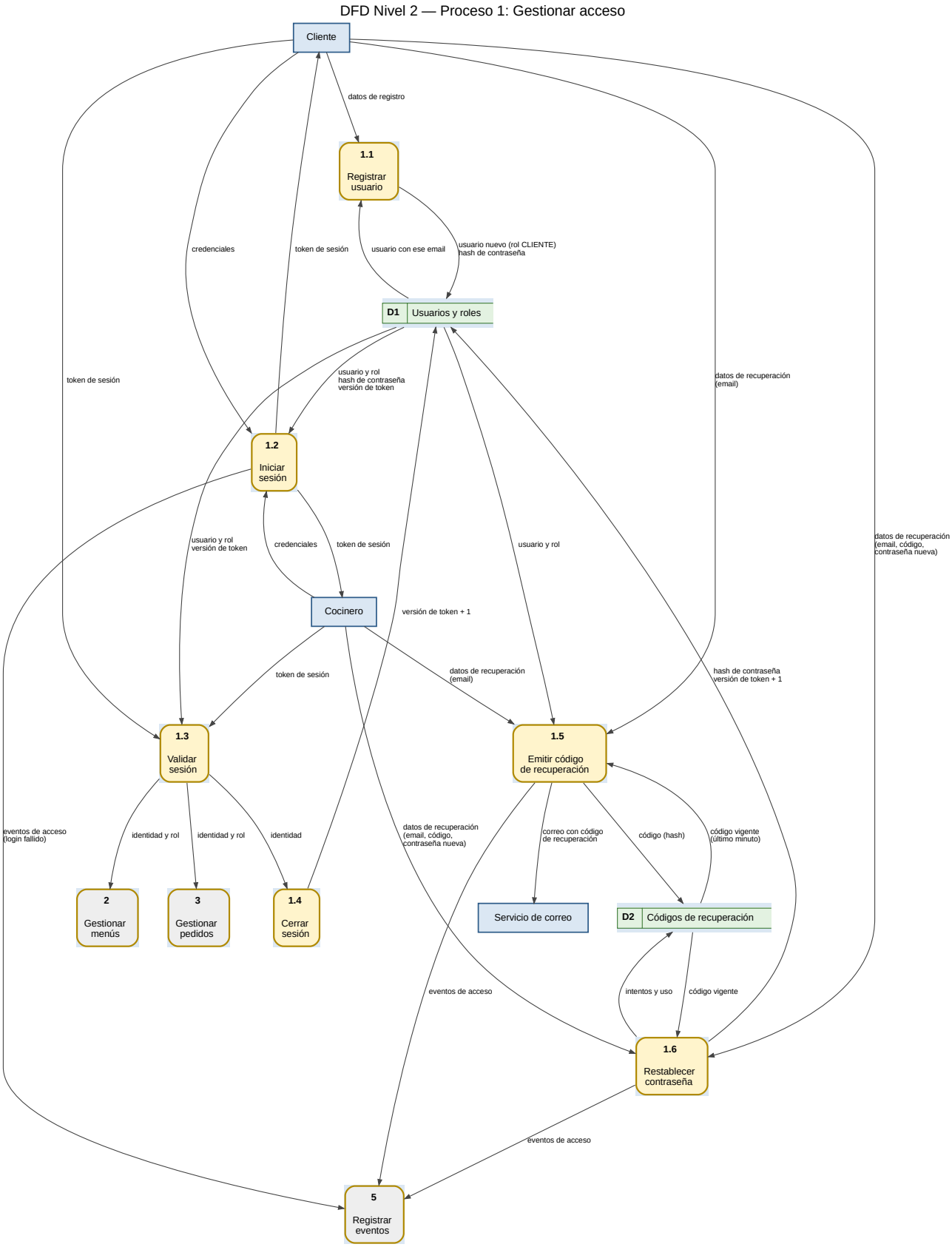
Cada flujo del diagrama de contexto aparece en el Nivel 1, con el mismo nombre, conectado al proceso que lo produce o lo consume. No hay en el Nivel 1 flujos con entidades externas que no estén en el Nivel 0.

Flujo del Nivel 0	Sentido	Proceso del Nivel 1
datos de registro	Cliente → sistema	1
credenciales	Cliente, Cocinero → sistema	1
token de sesión	Cliente, Cocinero → sistema	1
datos de recuperación	Cliente, Cocinero → sistema	1
token de sesión	sistema → Cliente, Cocinero	1
correo con código de recuperación	sistema → Servicio de correo	1
datos del menú · menú a eliminar	Cocinero → sistema	2
menús con viandas reservadas	sistema → Cocinero	2
menús con precio y disponibilidad	sistema → Cliente	2
datos del pedido · pedido a cancelar o pagar	Cliente → sistema	3
mis pedidos con estado de pago · URL de pago	sistema → Cliente	3
pedidos de los clientes por día	sistema → Cocinero	3
id de pago	Cliente → sistema	4
resultado del pago	sistema → Cliente	4
orden de pago · id de pago a consultar · pago a devolver	sistema → Mercado Pago	4
URL de pago · notificación de pago · estado y monto del pago	Mercado Pago → sistema	4
registro de eventos (JSON)	sistema → Plataforma de logs	5

Nivel 2 (consigna 4.3)

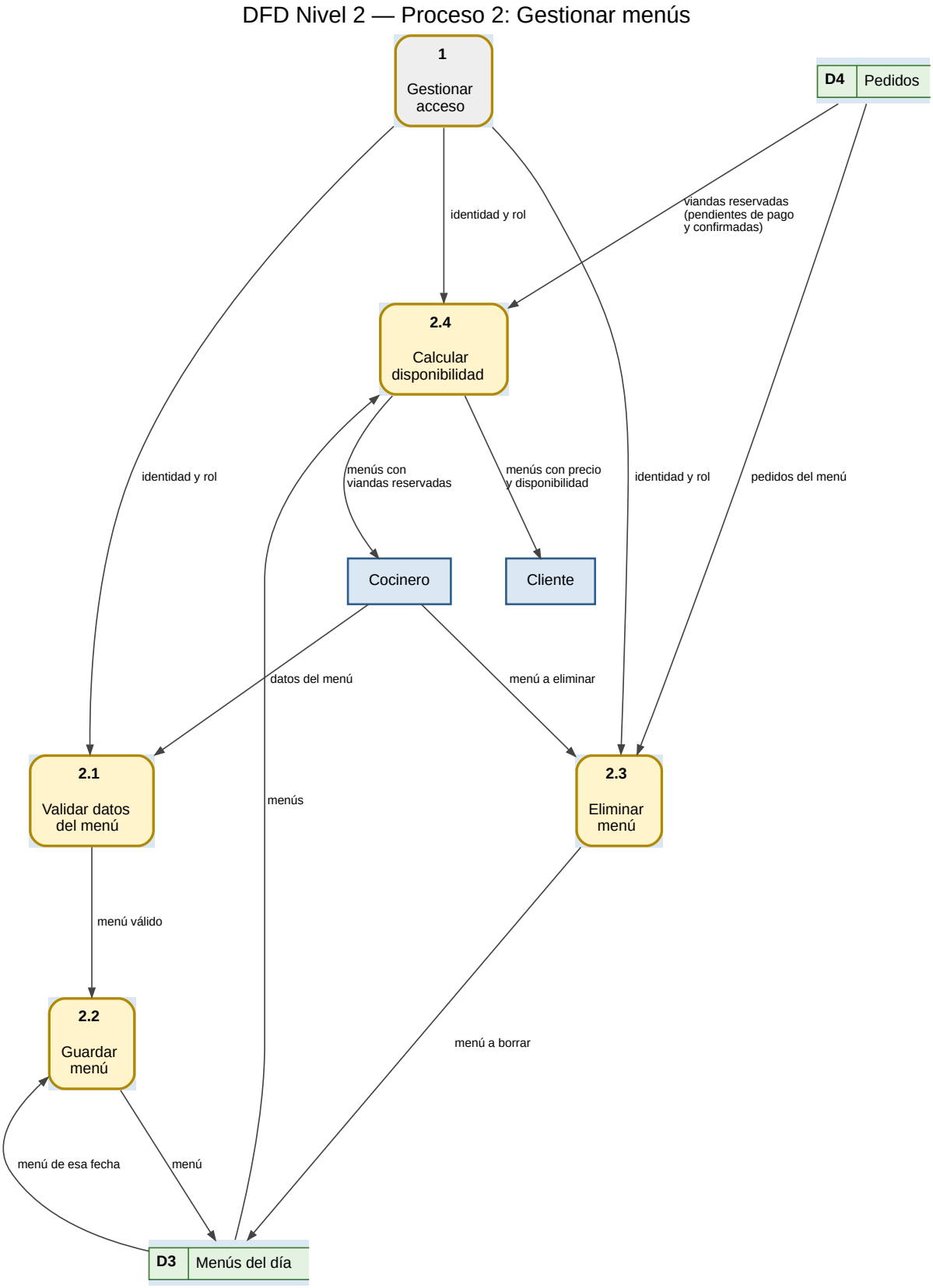
Se descomponen los cuatro procesos que manejan datos del negocio. La numeración es jerárquica y cada diagrama usa sólo los flujos que su proceso tiene en el Nivel 1 (en el proceso 3, “pedido a cancelar o pagar” se separa en “pedido a cancelar” y “pedido a pagar”). La descripción de cada subproceso está en el documento de su módulo.

Proceso 1 — Gestionar acceso



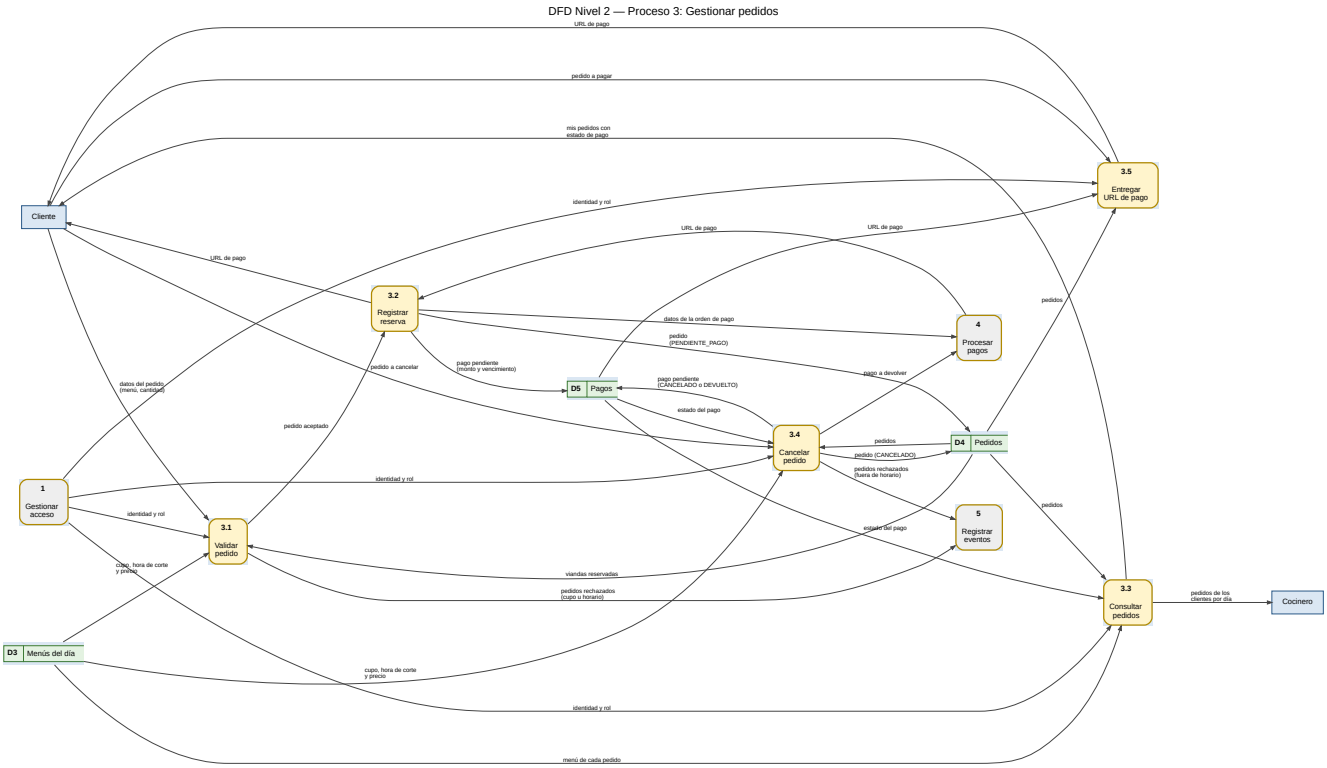
DFD Nivel 2 — proceso 1

Proceso 2 — Gestionar menús



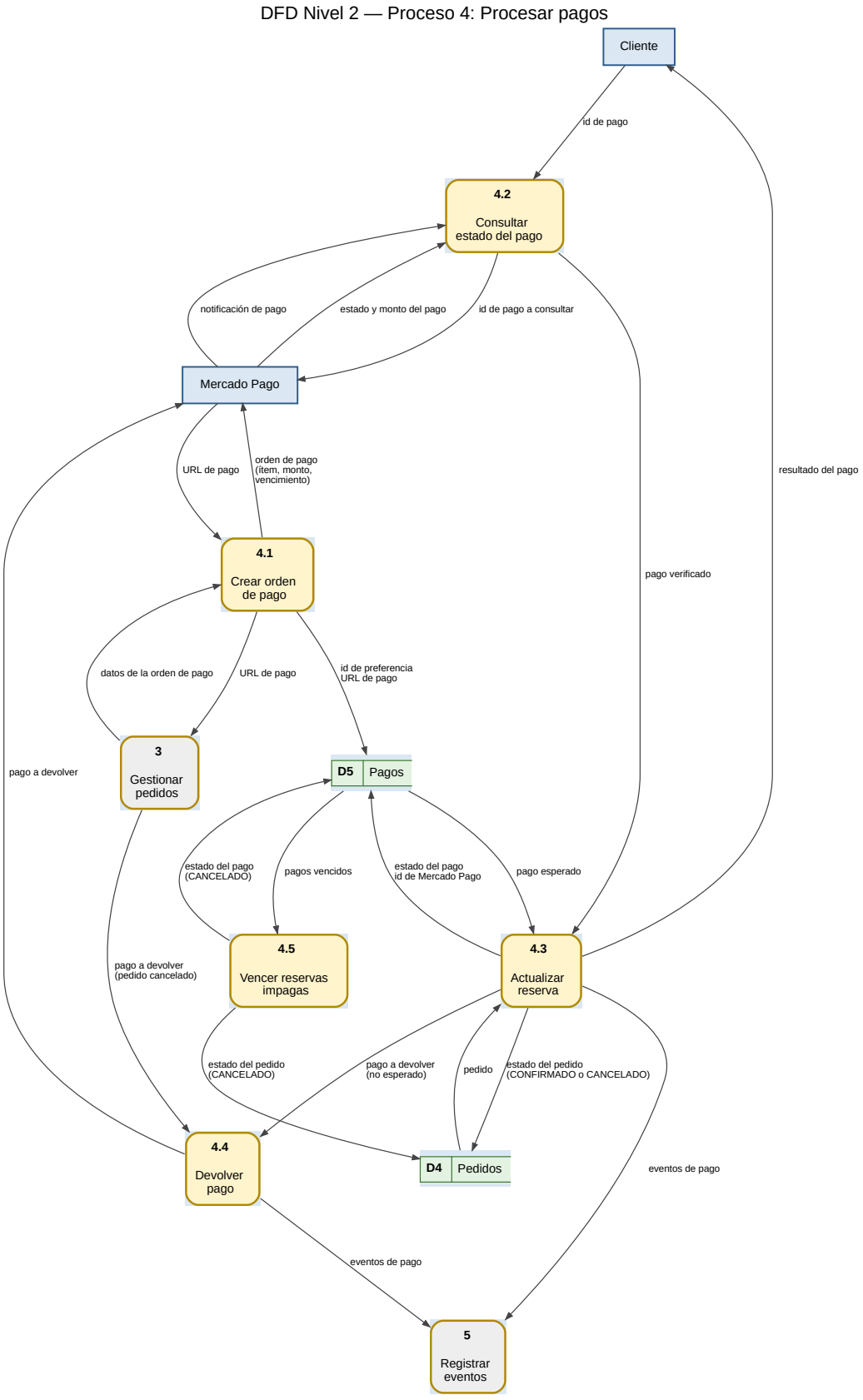
DFD Nivel 2 — proceso 2

Proceso 3 — Gestionar pedidos



DFD Nivel 2 — proceso 3

Proceso 4 — Procesar pagos



DFD Nivel 2 — proceso 4

Subproceso	Qué hace	Código
1.1 Registrar usuario	Valida, hashea la contraseña y crea un CLIENTE	`POST /api/auth/registro`
1.2 Iniciar sesión	Compara la contraseña y emite el JWT	`POST /api/auth/login`
1.3 Validar sesión	Verifica el token y lee el rol de la base	`autenticar`
1.4 Cerrar sesión	Sube `versionToken`	`POST /api/auth/logout`
1.5 Emitir código de recuperación	Genera el código, guarda su HMAC y lo manda por correo	`POST /api/auth/recuperar`
1.6 Restablecer contraseña	Verifica el código y guarda la contraseña nueva	`POST /api/auth/recuperar/verificar` y `/cambiar`
2.1 Validar datos del menú	zod + rol COCINERO	`esquemaMenu`, `exigirRol`
2.2 Guardar menú	Crea o modifica controlando la fecha única	`POST` / `PUT /api/menus`
2.3 Eliminar menú	Borra si no tiene pedidos	`DELETE /api/menus/:id`
2.4 Calcular disponibilidad	Suma reservas y calcula `disponibles` y `abierto`	`conTotales`
3.1 Validar pedido	Cupo y hora de corte, con la fila del menú bloqueada	`validarNuevoPedido`
3.2 Registrar reserva	Crea pedido y pago pendientes	`POST /api/pedidos`
3.3 Consultar pedidos	Lista según el rol, con `cancelable` y `pagable`	`GET /api/pedidos`
3.4 Cancelar pedido	Controla el corte y cancela (pide devolución si estaba pagado)	`PATCH /api/pedidos/:id/cancelar`
3.5 Entregar URL de pago	Devuelve la URL para reintentar	`POST /api/pedidos/:id/pagar`
4.1 Crear orden de pago	Crea la preferencia de Checkout Pro	`crearPreferencia`
4.2 Consultar estado del pago	Consulta `GET /v1/payments/{id}`	`obtenerPago`
4.3 Actualizar reserva	Confirma, deja pendiente o manda a devolver	`procesarPago`
4.4 Devolver pago	Pide la devolución total	`devolverPago`
4.5 Vencer reservas impagas	Cancela reservas con el plazo vencido	`vencerReservasImpagas`

Casos de uso (consigna 4.4)

1. Diagrama general

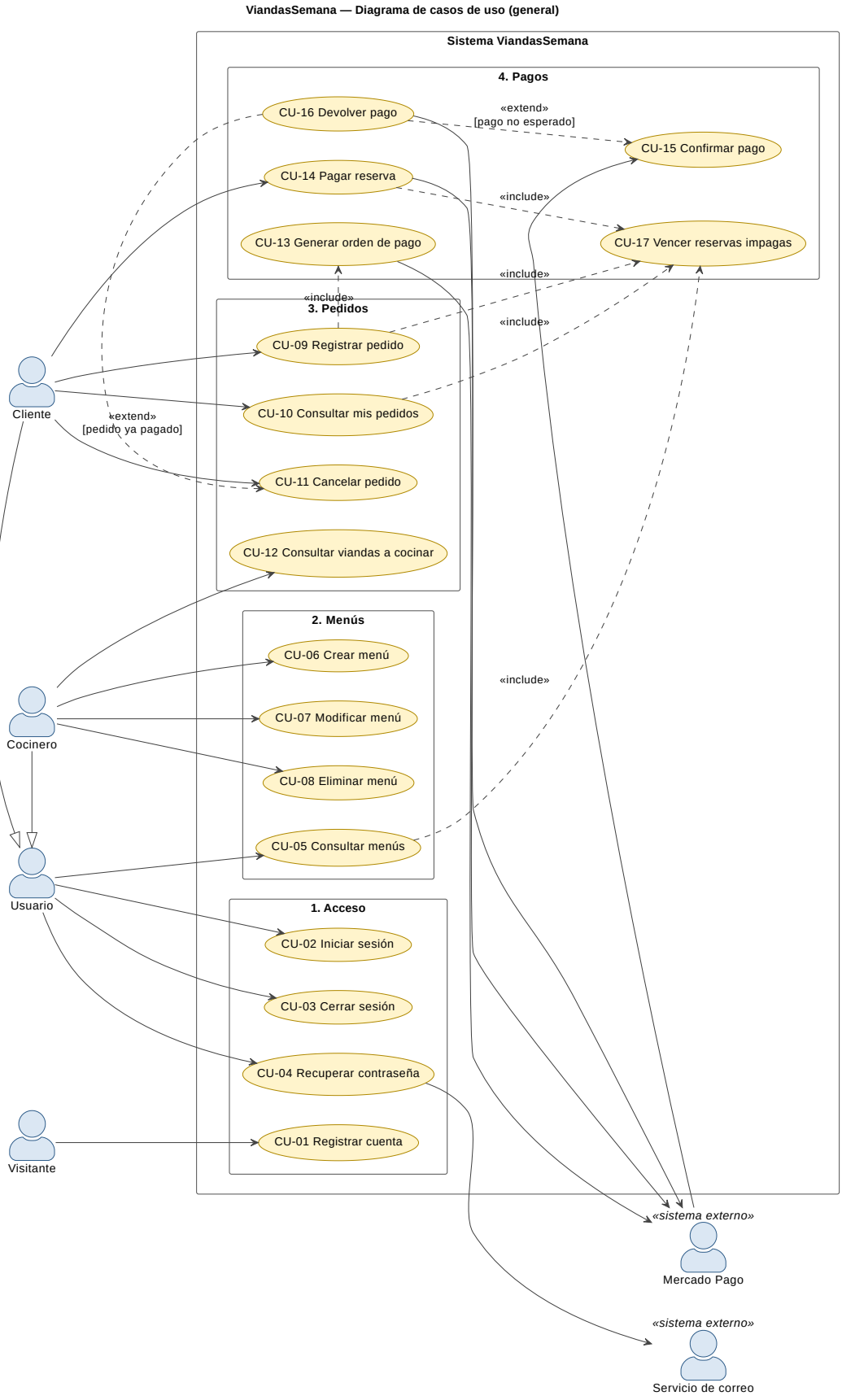


Diagrama general de casos de uso

2. Actores

Actor	Tipo	Participa en
Visitante	humano	CU-01
Usuario (generalización de Cliente y Cocinero)	humano	CU-02, CU-03, CU-04, CU-05
Cliente	humano	CU-09, CU-10, CU-11, CU-14 (y los de Usuario)
Cocinero	humano	CU-06, CU-07, CU-08, CU-12 (y los de Usuario)
Mercado Pago	sistema externo	CU-13, CU-14, CU-15 (actor principal: avisa los pagos por webhook), CU-16
Servicio de correo	sistema externo	CU-04

No hay procesos programados: el vencimiento de reservas impagas (CU-17) se ejecuta incluido en otros casos de uso, justo antes de contar el cupo (ver la justificación en el módulo 4).

3. Catálogo por módulo

Módulo	Caso de uso	Actor principal	Endpoint
1 Acceso	CU-01 Registrar cuenta	Visitante	`POST /api/auth/registro`
	CU-02 Iniciar sesión	Usuario	`POST /api/auth/login`
	CU-03 Cerrar sesión	Usuario	`POST /api/auth/logout`
	CU-04 Recuperar contraseña	Usuario	`POST /api/auth/recuperar`, `/verificar`, `/cambiar`
2 Menús	CU-05 Consultar menús	Usuario	`GET /api/menus`, `GET /api/menus/:id`
	CU-06 Crear menú	Cocinero	`POST /api/menus`
	CU-07 Modificar menú	Cocinero	`PUT /api/menus/:id`
	CU-08 Eliminar menú	Cocinero	`DELETE /api/menus/:id`
3 Pedidos	**CU-09 Registrar pedido** ★	Cliente	`POST /api/pedidos`
	CU-10 Consultar mis pedidos	Cliente	`GET /api/pedidos`
	CU-11 Cancelar pedido ★	Cliente	`PATCH /api/pedidos/:id/cancelar`
	CU-12 Consultar viandas a cocinar	Cocinero	`GET /api/pedidos`
4 Pagos	CU-13 Generar orden de pago	(incluido en CU-09)	Mercado Pago `POST /checkout/preferences`
	CU-14 Pagar reserva	Cliente	`POST /api/pedidos/:id/pagar`
	CU-15 Confirmar pago ★	Mercado Pago	`POST /api/pagos/notificacion`

Módulo	Caso de uso	Actor principal	Endpoint
	CU-16 Devolver pago	(extiende CU-11 y CU-15)	Mercado Pago `POST /v1/payments/{id}/refunds`
	CU-17 Vencer reservas impagas	(incluido en CU-05, CU-09, CU-10, CU-14)	—

★ = caso de uso principal (su narrativa está abajo). Las plantillas completas de **todos** los casos de uso están en el documento de cada módulo: Acceso · Menús · Pedidos · Pagos.

4. Relaciones `<<include>>` y `<<extend>>`

Relación	Por qué
CU-09 `<<include>>` CU-13 Generar orden de pago	Toda reserva genera siempre su orden de pago: sin ella la reserva se cancela
CU-05, CU-09, CU-10, CU-14 `<<include>>` CU-17 Vencer reservas impagas	Comportamiento común que se ejecuta **siempre** antes de mostrar o contar el cupo
CU-16 Devolver pago `<<extend>>` CU-11 Cancelar pedido	Sólo ocurre si el pedido cancelado **ya estaba pagado**
CU-16 Devolver pago `<<extend>>` CU-15 Confirmar pago	Sólo ocurre si llega un pago aprobado **no esperado** (reserva vencida o cancelada, monto distinto, pago doble)
Cliente y Cocinero son un Usuario (generalización)	Los dos inician y cierran sesión, recuperan la contraseña y consultan menús

No se usa `<<include>>` para “validar sesión”: es una precondición de todos los casos de uso con sesión y dibujarla en cada uno no agrega información.

5. Narrativas de los tres casos de uso principales

Son los que sostienen el negocio: reservar con cupo y horario (CU-09), confirmar la reserva sólo con el pago real (CU-15) y cancelar con devolución (CU-11). Son copia de las plantillas de sus módulos.

CU-09 Registrar pedido ★ (caso de uso principal)

Campo	Detalle
Módulo	3 — Pedidos
Actor principal	Cliente
Actores secundarios	Mercado Pago (a través de CU-13)
Descripción	El cliente reserva una o más viandas de un menú; la reserva queda pendiente de pago y el cliente pasa a pagar en Mercado Pago.
Precondiciones	Sesión iniciada con rol `CLIENTE`. El menú existe, tiene precio y está abierto.
Disparador	El cliente elige la cantidad y toca “Reservar y pagar” en la tarjeta de un menú.
Relaciones	`<<include>>` CU-17 Vencer reservas impagas · `<<include>>` CU-13 Generar orden de pago

Campo	Detalle
Endpoint	`POST /api/pedidos { menuId, cantidad }` → `201 { ...pedido, pago, urlPago }`

Flujo principal

1. El cliente indica la cantidad y toca “Reservar y pagar”.
2. El frontend envía `POST /api/pedidos` con el token.
3. La API valida la sesión y que el rol sea `CLIENTE`.
4. La API valida los datos con zod (`menuId` entero positivo, `cantidad` entero ≥ 1).
5. La API abre una transacción y **bloquea la fila del menú** (`SELECT … FOR UPDATE`): si llegan dos pedidos a la vez, el segundo espera al primero.
6. La API verifica que el menú tenga precio.
7. La API cancela las reservas impagas vencidas (CU-17) para que no ocupen cupo.
8. La API suma las viandas de los pedidos `PENDIENTE_PAGO` y `CONFIRMADO` del menú.
9. La API aplica `validarNuevoPedido`: la hora de corte no pasó y la cantidad no supera las viandas disponibles.
10. La API crea el pedido (`PENDIENTE_PAGO`) y su pago (`PENDIENTE`, `monto = cantidad × precio`, vencimiento = 30 minutos o la hora de corte, lo que ocurra antes) y confirma la transacción.
11. La API genera la orden de pago en Mercado Pago (CU-13), guarda su id y la URL de pago.
12. La API responde `201` con el pedido, el pago y `urlPago`.
13. El frontend avisa “Te llevamos a Mercado Pago para pagar tu reserva…” y redirige al cliente a `urlPago` (continúa en CU-14).

Flujos alternativos

- **3a. Sin sesión:** `401`. **3b. Rol COCINERO:** `403`.
- **4a. Datos inválidos:** `400` con el detalle por campo.
- **5a. El menú no existe:** `404 "Menú no encontrado"`; se deshace la transacción.
- **6a. Menú sin precio:** `409 SIN_PRECIO "Este menú todavía no tiene precio"`.
- **9a. Pasó la hora de corte:** `409 FUERA_DE_HORARIO`; se registra el evento `regla_negocio` en el log.
- **9b. No alcanza el cupo:** `409 CUPO_COMPLETO` con “No quedan viandas disponibles para este día” o “Sólo quedan N viandas”; se registra `regla_negocio`.
- **11a. Mercado Pago no responde, falla o no está configurado:** el pedido y su pago pasan a `CANCELADO` (libera el cupo) y la API responde `502`/`503` con “Probá de nuevo en unos minutos”; el error queda en el log.

Postcondiciones

- Éxito: existe un `Pedido` `PENDIENTE_PAGO` con su `Pago` `PENDIENTE` y la URL de pago; las viandas quedan apartadas del cupo hasta que se pague, se cancele o venza.
- Fracaso: no queda ninguna reserva ocupando cupo.

Reglas de negocio: cupo máximo por día; no se pide después de la hora de corte; dos pedidos simultáneos nunca superan el cupo (bloqueo de fila + transacción).

CU-15 Confirmar pago ★ (caso de uso principal)

Campo	Detalle
Módulo	4 — Pagos
Actor principal	Mercado Pago (webhook). También lo dispara el frontend cuando el cliente vuelve de pagar.

Campo	Detalle
Actores secundarios	Cliente (recibe el resultado)
Descripción	El sistema consulta el estado real de un pago y actualiza la reserva: la confirma, la deja pendiente o devuelve el dinero.
Precondiciones	Existe un pago en Mercado Pago cuyo `external_reference` es el id de un pedido.
Disparador	`POST /api/pagos/notificacion` con `{ type: "payment", data: { id } }` (o los mismos datos en la URL).
Relaciones	Extendido por CU-16 Devolver pago `[pago no esperado]`
Endpoint	`POST /api/pagos/notificacion` → `200 { pedidoId, estadoPedido, estadoPago }` (público: no lleva token)

Flujo principal

1. Mercado Pago (o el frontend, al volver el cliente) envía el aviso con el id del pago.
2. La API ignora los avisos que no son de tipo `payment` (responde `200 { ignorado: true }`).
3. La API valida que el id sea numérico.
4. La API consulta `GET /v1/payments/{id}` a Mercado Pago: estado, monto, moneda y `external_reference`.
5. La API traduce el estado (`approved` → `APROBADO`, `rejected` → `RECHAZADO`, `cancelled` → `CANCELADO`, `refunded` / `charged_back` → `DEVUELTO`, el resto → `PENDIENTE`).
6. En una transacción, bloquea la fila del pedido (`FOR UPDATE`) para procesar de a un aviso.
7. Si el aviso ya se procesó (mismo id y mismo estado), no cambia nada (idempotente).
8. Si el pago está `APROBADO`, el pedido está `PENDIENTE_PAGO` y el monto y la moneda (ARS) coinciden: el pago pasa a `APROBADO` con el id de Mercado Pago y el pedido a `CONFIRMADO`; se registra `pago_aprobado`.
9. La API responde el estado del pedido y del pago; el frontend muestra “¡Pago aprobado! Tu reserva está confirmada.” y abre “Mis pedidos”.

Flujos alternativos

- **2a. Aviso de otro tipo (por ejemplo `merchant_order`):** `200 { ignorado: true }`, para que Mercado Pago no lo reintente.
- **3a. Id inválido:** `400`.
- **4a. El pago no existe en Mercado Pago:** `404`. **4b. Mercado Pago no responde:** `502` (Mercado Pago reintenta el webhook más tarde).
- **6a. El pago no corresponde a ninguna reserva:** `404 "El pago no corresponde a ninguna reserva"`.
- **8a. Pago pendiente, rechazado o cancelado:** el pago guarda ese estado y el pedido **sigue** `PENDIENTE_PAGO`; el cliente puede reintentar hasta que venza. El frontend muestra “El pago no se aprobó…” o “Mercado Pago está procesando el pago…”.
- **8b. Pago aprobado pero la reserva ya estaba cancelada o vencida, el monto no coincide o la reserva ya estaba pagada con otro pago:** se ejecuta CU-16 Devolver pago y se registra `pago_devuelto`. Si el pedido no estaba confirmado, el pago queda `DEVUELTO`.
- **8c. Devolución o contracargo hecho desde Mercado Pago sobre una reserva confirmada:** el pago pasa a `DEVUELTO` y el pedido a `CANCELADO` (libera el cupo).

Postcondiciones: el estado de la reserva coincide con el estado real del pago en Mercado Pago. Ninguna reserva se confirma sin un pago aprobado por el monto exacto.

Reglas: Mercado Pago es la única fuente de verdad del pago; repetir el aviso no cambia nada; dos avisos simultáneos del mismo pago se procesan de a uno.

CU-11 Cancelar pedido ★ (caso de uso principal)

Campo	Detalle
Módulo	3 — Pedidos
Actor principal	Cliente
Actores secundarios	Mercado Pago (si hay que devolver el dinero)
Descripción	El cliente cancela una reserva propia antes de la hora de corte; si ya estaba pagada, se le devuelve el dinero.
Precondiciones	Sesión con rol `CLIENTE`; el pedido es del cliente y no está cancelado.
Disparador	El cliente toca “Cancelar” en “Mis pedidos” y confirma.
Relaciones	Extendido por CU-16 Devolver pago `[pedido ya pagado]`
Endpoint	`PATCH /api/pedidos/:id/cancelar` → `200`

Flujo principal

1. El frontend pide confirmación; si el pedido está pagado agrega “Te devolvemos el pago por Mercado Pago”.
2. El frontend envía `PATCH /api/pedidos/:id/cancelar`.
3. La API valida la sesión, el rol `CLIENTE` y que el id sea un entero positivo.
4. La API abre una transacción y **bloquea la fila del pedido** (`FOR UPDATE`) para que no se confirme un pago mientras se cancela.
5. La API verifica que el pedido exista y sea del usuario del token.
6. La API aplica `validarCancelacion`: el pedido no está cancelado y no pasó la hora de corte del menú.
7. Si el pago estaba `PENDIENTE` o `RECHAZADO`, lo pasa a `CANCELADO`.
8. La API pasa el pedido a `CANCELADO` (libera el cupo) y lo devuelve.
9. El frontend muestra “Pedido cancelado” y recarga la lista.

Flujos alternativos

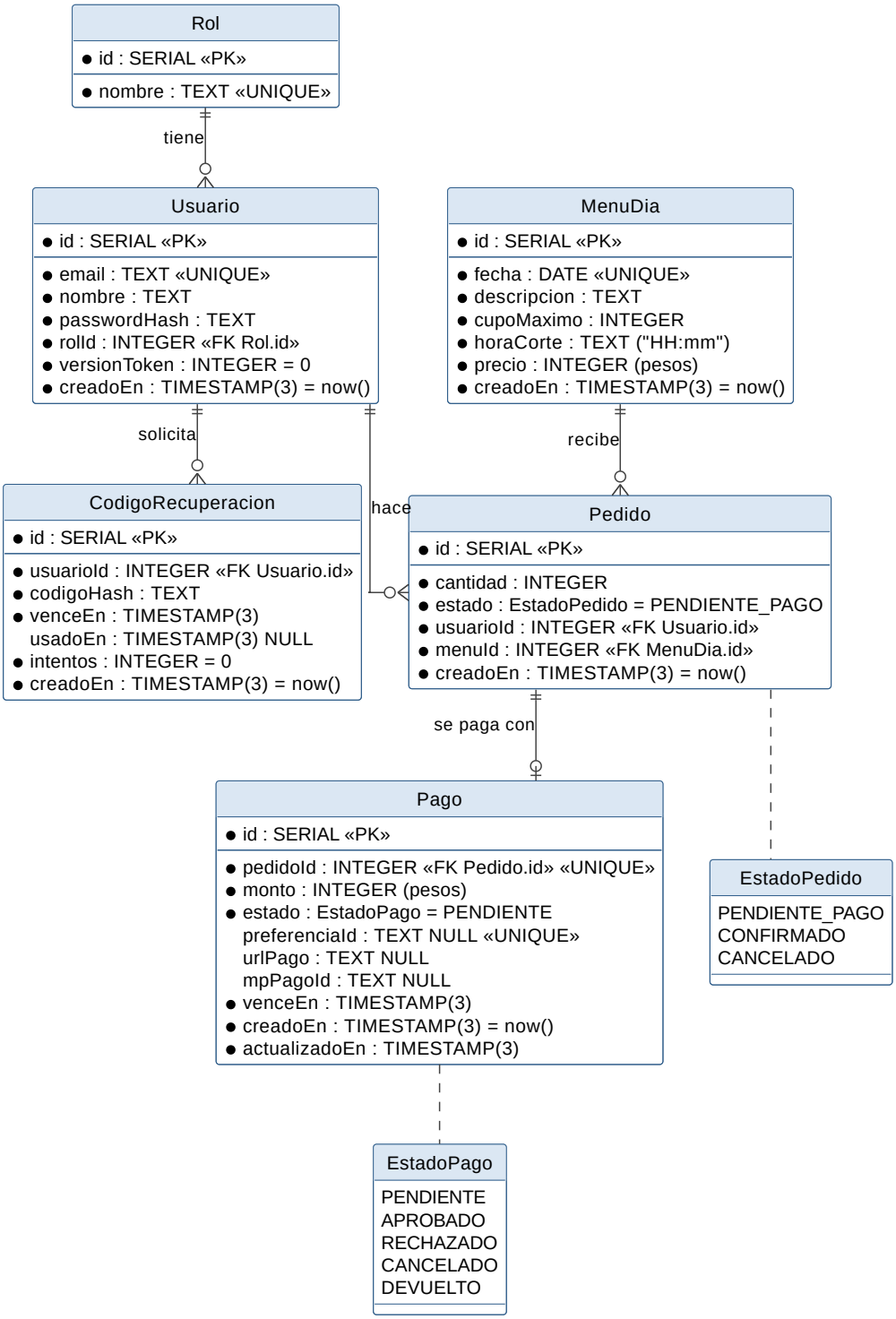
- **3a. Rol COCINERO:** `403`. **3b. Id inválido:** `400`.
- **5a. El pedido no existe o es de otro cliente:** `404 "Pedido no encontrado"` (no revela que existe).
- **6a. Ya estaba cancelado:** `409 YA_CANCELADO`.
- **6b. Pasó la hora de corte:** `409 FUERA_DE_HORARIO` (“No se puede cancelar un pedido después del horario de corte”).
- **7a. El pedido estaba pagado (`APROBADO`):** se ejecuta CU-16 Devolver pago; el pago queda `DEVUELTO` y se registra `pago_devuelto`.
- **7b. Mercado Pago falla al devolver:** la transacción se deshace, el pedido **no** se cancela y la API responde `502`.

Postcondiciones: el pedido queda `CANCELADO`, sus viandas vuelven al cupo y, si estaba pagado, el dinero se devolvió.

Modelo de datos — DER (consigna 4.5)

La fuente de verdad es `backend/prisma/schema.prisma`; las tablas se crean con las migraciones de `backend/prisma/migrations/` (`20261004183355_inicial` y `20261004200000_recuperacion_y_pagos`). El DER refleja exactamente lo implementado en PostgreSQL: nombres de tablas y columnas, tipos SQL, claves y restricciones.

ViandasSemana — Modelo de datos (DER)
Fuente de verdad: backend/prisma/schema.prisma



DER

Cardinalidades

Relación	Cardinalidad	Clave foránea	Al borrar el padre
Rol — Usuario	1 : N (un rol tiene 0..N usuarios; un usuario tiene exactamente 1 rol)	`Usuario.rolId → Rol.id`	RESTRICT
Usuario — CodigoRecuperacion	1 : N (0..N códigos por usuario)	`CodigoRecuperacion.usuarioId → Usuario.id`	CASCADE
Usuario — Pedido	1 : N (0..N pedidos por usuario)	`Pedido.usuarioId → Usuario.id`	RESTRICT
MenuDia — Pedido	1 : N (0..N pedidos por menú)	`Pedido.menuId → MenuDia.id`	RESTRICT
Pedido — Pago	1 : 0..1 (un pedido tiene a lo sumo un pago)	`Pago.pedidoId → Pedido.id` (UNIQUE)	CASCADE

Diccionario de datos

Rol

Columna	Tipo	Restricciones	Descripción
`id`	SERIAL	PK	
`nombre`	TEXT	NOT NULL, UNIQUE	`COCINERO` o `CLIENTE` (los inserta la migración inicial)

Usuario

Columna	Tipo	Restricciones	Descripción
`id`	SERIAL	PK	
`email`	TEXT	NOT NULL, UNIQUE	Se guarda en minúsculas
`nombre`	TEXT	NOT NULL	
`passwordHash`	TEXT	NOT NULL	Hash bcrypt (incluye la sal). Nunca la contraseña
`rolId`	INTEGER	NOT NULL, FK → `Rol.id`	
`versionToken`	INTEGER	NOT NULL, DEFAULT 0	Sube en cada logout y al cambiar la contraseña: invalida los tokens anteriores
`creadoEn`	TIMESTAMP(3)	NOT NULL, DEFAULT now	

CodigoRecuperacion

Columna	Tipo	Restricciones	Descripción
`id`	SERIAL	PK	
`usuarioId`	INTEGER	NOT NULL, FK → `Usuario.id`, índice	

Columna	Tipo	Restricciones	Descripción
`codigoHash`	TEXT	NOT NULL	HMAC-SHA256 del código de 6 dígitos. Nunca el código
`venceEn`	TIMESTAMP(3)	NOT NULL	15 minutos después de creado
`usadoEn`	TIMESTAMP(3)	NULL	Se completa al usarlo, al reemplazarlo o al bloquearlo
`intentos`	INTEGER	NOT NULL, DEFAULT 0	Intentos fallidos (máximo 5)
`creadoEn`	TIMESTAMP(3)	NOT NULL, DEFAULT now	Se usa para el límite de un código por minuto

MenuDia

Columna	Tipo	Restricciones	Descripción
`id`	SERIAL	PK	
`fecha`	DATE	NOT NULL, UNIQUE	Un menú por día
`descripcion`	TEXT	NOT NULL	
`cupoMaximo`	INTEGER	NOT NULL	Viandas que se pueden cocinar ese día (≥ 1, lo valida la API)
`horaCorte`	TEXT	NOT NULL	“HH:mm”, hora de Argentina
`precio`	INTEGER	NOT NULL	Pesos por vianda (≥ 1 al crear; 0 en menús anteriores a los pagos)
`creadoEn`	TIMESTAMP(3)	NOT NULL, DEFAULT now	

Pedido

Columna	Tipo	Restricciones	Descripción
`id`	SERIAL	PK	
`cantidad`	INTEGER	NOT NULL	Viandas reservadas (≥ 1)
`estado`	`EstadoPedido`	NOT NULL, DEFAULT `PENDIENTE_PAGO`	`PENDIENTE_PAGO`, `CONFIRMADO` o `CANCELADO`
`usuarioId`	INTEGER	NOT NULL, FK → `Usuario.id`	Cliente que reservó
`menuId`	INTEGER	NOT NULL, FK → `MenuDia.id`	
`creadoEn`	TIMESTAMP(3)	NOT NULL, DEFAULT now	

Pago

Columna	Tipo	Restricciones	Descripción
`id`	SERIAL	PK	
`pedidoId`	INTEGER	NOT NULL, UNIQUE, FK → `Pedido.id`	
`monto`	INTEGER	NOT NULL	Pesos: cantidad × precio
`estado`	`EstadoPago`	NOT NULL, DEFAULT `PENDIENTE`	`PENDIENTE`, `APROBADO`, `RECHAZADO`, `CANCELADO` o `DEVUELTO`
`preferenciaId`	TEXT	NULL, UNIQUE	Preferencia de Checkout Pro
`urlPago`	TEXT	NULL	`init_point` de la preferencia
`mpPagoId`	TEXT	NULL	Id del pago en Mercado Pago
`venceEn`	TIMESTAMP(3)	NOT NULL	Hasta cuándo se puede pagar
`creadoEn`	TIMESTAMP(3)	NOT NULL, DEFAULT now	
`actualizadoEn`	TIMESTAMP(3)	NOT NULL	Lo actualiza Prisma en cada cambio

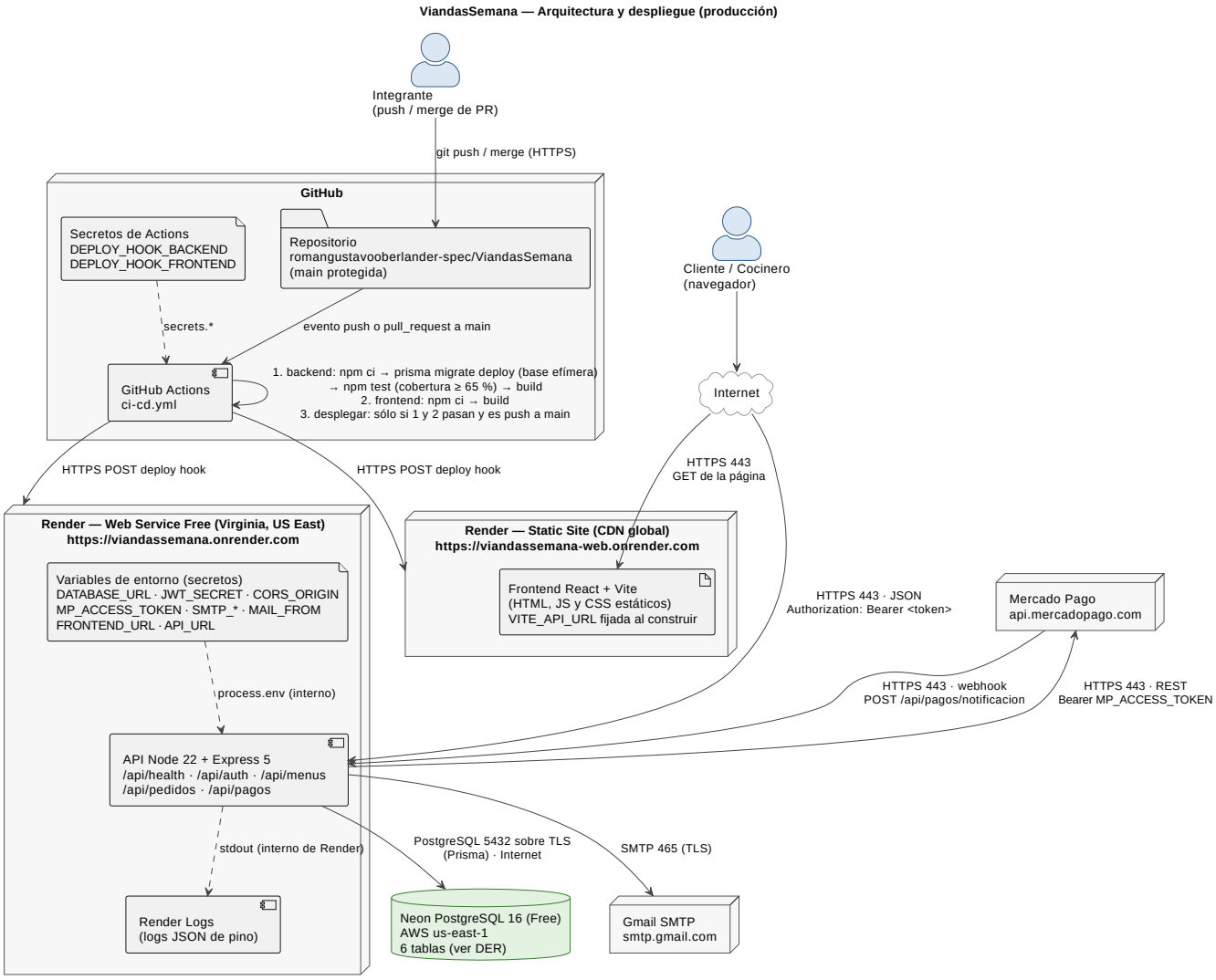
Consultas para mostrar en vivo

```
        -- Usuarios con su rol (la contraseña no se ve: sólo el hash)
        SELECT u.id, u.email, r.nombre AS rol, left(u."passwordHash", 20) AS hash FROM "Usuario" u
JOIN "Rol" r ON r.id = u."rolId";

        -- Viandas reservadas por menú (lo que ocupa cupo)
        SELECT m.fecha, m."cupoMaximo", COALESCE(SUM(p.cantidad) FILTER (WHERE p.estado IN
('PENDIENTE_PAGO','CONFIRMADO')), 0) AS reservadas
        FROM "MenuDia" m LEFT JOIN "Pedido" p ON p."menuId" = m.id GROUP BY m.id ORDER BY m.fecha;

        -- Pedidos con su pago
        SELECT p.id, p.estado, p.cantidad, g.estado AS pago, g.monto, g."mpPagoId" FROM "Pedido" p
LEFT JOIN "Pago" g ON g."pedidoId" = p.id ORDER BY p.id DESC;
```

Arquitectura y despliegue (consigna 4.6)



Arquitectura y despliegue en producción

1. Componentes desplegables

Componente	Tecnología	Plataforma	URL productiva
Cliente (frontend)	React 18 + Vite, sitio estático	Render Static Site (CDN global)	https://viandassemana-web.onrender.com
API (backend)	Node 22 + Express 5 + TypeScript + Prisma	Render Web Service, plan Free, Virginia (US East)	https://viandassemana.onrender.com/api/health
Base de datos	PostgreSQL 16	Neon, plan Free, AWS us-east-1	(privada: sólo con `DATABASE_URL`)
Pagos	Mercado Pago Checkout Pro	servicio externo	https://api.mercadopago.com
Correo	SMTP de Gmail (cuenta del proyecto)	servicio externo	smtp.gmail.com

Componente	Tecnología	Plataforma	URL productiva
Logs	Render Logs	Render	panel de Render del Web Service
Repositorio y CI/CD	GitHub + GitHub Actions	GitHub	https://github.com/romangustavooberlander-spec/ViandasSemana

No hay caché ni cola de mensajes. El plan gratuito de Render duerme la API tras 15 minutos sin tráfico; la primera petición posterior tarda alrededor de un minuto.

2. Comunicaciones

Desde → hacia	Protocolo y puerto	¿Atraviesa Internet?
Navegador → frontend (Render Static Site)	HTTPS, 443	Sí
Navegador → API	HTTPS, 443, JSON, `Authorization: Bearer <token>`	Sí
API → Neon PostgreSQL	protocolo PostgreSQL sobre TLS, 5432 (Prisma)	Sí (Render y Neon son proveedores distintos)
API → Mercado Pago	HTTPS, 443, REST con `Bearer MP_ACCESS_TOKEN`	Sí
Mercado Pago → API (webhook)	HTTPS, 443, `POST /api/pagos/notificacion`	Sí
API → Gmail	SMTP con TLS, 465	Sí
API → Render Logs	salida estándar del proceso	No (interno de Render)
Render → API	el balanceador de Render termina el HTTPS y reenvía al puerto de la variable `PORT`	No (interno de Render)
GitHub Actions → Render	HTTPS, 443, POST a los deploy hooks	Sí

El frontend nunca habla con la base de datos: todo pasa por la API (consigna 3.1).

3. Secretos y su recorrido

Secreto	Dónde se carga	Cómo llega al componente
`DATABASE_URL`, `JWT_SECRET`, `MP_ACCESS_TOKEN`, `SMTP_USER`, `SMTP_PASS`	Variables de entorno del Web Service en el panel de Render	Render las pone en `process.env` al iniciar → `backend/src/config/env.ts` (única lectura) → Prisma, `jsonwebtoken`, `lib/mercadoPago.ts`, `lib/mailer.ts`
`CORS_ORIGIN`, `FRONTEND_URL`, `API_URL`, `SMTP_HOST`, `SMTP_PORT`, `MAIL_FROM`	Ídem (configuración, no secretos)	Ídem
`VITE_API_URL`	Variable de entorno del Static Site en Render	Vite la escribe en el JavaScript **al construir** (es pública: sólo es la URL de la API)
`DEPLOY_HOOK_BACKEND`, `DEPLOY_HOOK_FRONTEND`	Secretos de GitHub Actions (Settings → Secrets and variables → Actions)	`${{ secrets.* }}` en el job `desplegar` de `ci-cd.yml`

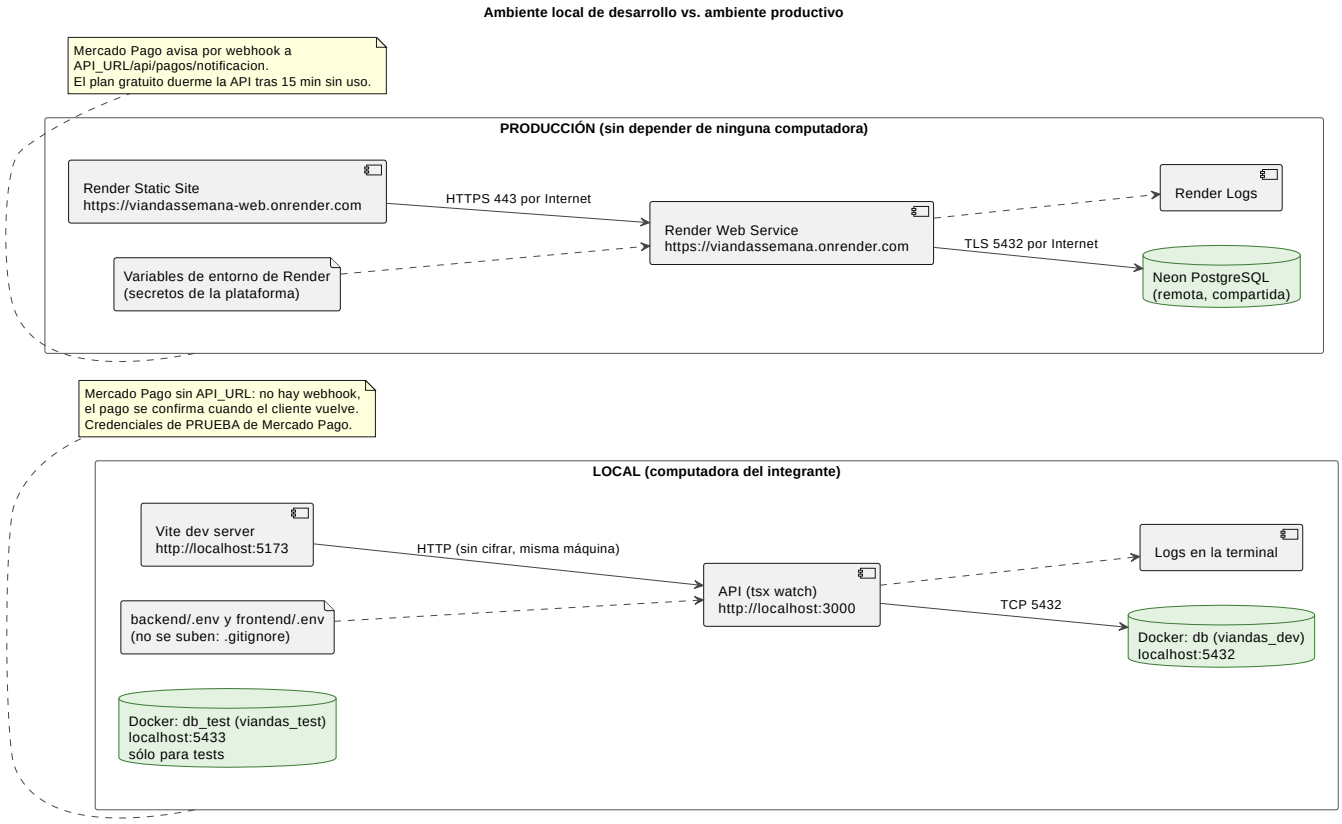
Secreto	Dónde se carga	Cómo llega al componente
Credenciales de la base de **tests**	Fijas en `vitest.config.ts` y `ci-cd.yml`	No son secretos: la base es efímera y local

Ningún secreto está en el repositorio: `.env` está en `.gitignore` y `.env.example` sólo tiene nombres. Los logs nunca incluyen el encabezado `Authorization` ni contraseñas (`redact` en `lib/logger.ts`).

4. Del push a producción

1. Un integrante sube una rama y abre un pull request a `main` (la rama `main` está protegida).
2. **Evento `pull_request`:** GitHub Actions corre los jobs `backend` y `frontend` (no despliega). El PR muestra si pasaron.
3. Se hace merge del PR. **Evento `push` a `main`:** se vuelve a correr el pipeline:
   - `backend`: `npm ci` → `npx prisma generate` → `npx prisma migrate deploy` contra una base PostgreSQL efímera del runner → `npm test` (tests unitarios + integración, falla si la cobertura de líneas es menor al 65 %) → sube el informe de cobertura → `npm run build`.
   - `frontend`: `npm ci` → `npm run build` (`tsc` + `vite build`).
4. **Sólo si los dos jobs pasaron**, el job `desplegar` hace `POST` a los deploy hooks de Render.
5. **Evento deploy hook:** Render descarga el commit de `main`, ejecuta el comando de build configurado en su panel y reemplaza la versión en línea (API y sitio estático).
6. El historial de ejecuciones queda en la pestaña **Actions** del repositorio (exitosas y fallidas, con el paso que falló).

5. Ambiente local vs. productivo



Ambiente local y productivo

Aspecto	Local	Producción
Frontend	`npm run dev`, http://localhost:5173	Render Static Site, HTTPS
API	`npm run dev` (`tsx watch`), http://localhost:3000	Render Web Service (`npm start`), HTTPS
Base de datos	Docker: `viandas_dev` en `localhost:5432`	Neon PostgreSQL remota y compartida
Base de tests	Docker: `viandas_test` en `localhost:5433`	Contenedor efímero en GitHub Actions
Configuración	archivos `.env` (no se suben)	variables de entorno de Render y secretos de GitHub
Mercado Pago	credenciales de prueba; sin `API_URL` no hay webhook y el pago se confirma cuando el cliente vuelve	`API_URL` pública: llega el webhook aunque el cliente cierre la pestaña
Logs	en la terminal	Render Logs
Cifrado	HTTP sin cifrar (misma máquina)	HTTPS con certificado válido de Render

La diferencia entre ambientes se resuelve **sólo por configuración**: el mismo código corre en los dos.

Decisiones técnicas (consigna 4.7)

Criterios usados en todas las decisiones: **costo** (todo debía ser gratuito), **curva de aprendizaje** para dos integrantes, **calidad de la documentación** y **adecuación al problema** (pocas pantallas, reglas de cupo y horario que tienen que cumplirse aun con pedidos simultáneos, y cobro en pesos).

1. Frontend — React 18 + Vite (TypeScript), sin librerías de UI ni router. *Alternativas:* Angular; Vue 3; Next.js. *Fundamento:* React tiene la documentación y la cantidad de ejemplos más grandes y ya lo conocíamos; Vite construye un sitio estático que se publica gratis en un CDN y arranca aunque la API no esté (consigna 3.1). Angular exige aprender mucho más (módulos, RxJS) para cuatro pantallas; Next.js agrega renderizado en el servidor, que no necesitamos y complica el despliegue estático. *Se resigna:* las herramientas que trae Angular "de fábrica" (formularios, router) y el SEO del renderizado en servidor; las pestañas se manejan con estado.

2. Backend — Node 22 + Express 5 + TypeScript, validación con zod. *Alternativas:* Python con FastAPI; Java con Spring Boot. *Fundamento:* el mismo lenguaje que el frontend (los tipos se entienden de los dos lados); Express es mínimo y muy documentado; Express 5 lleva solos los errores de las rutas `async` al manejador central; zod da errores 400 claros por campo. Spring Boot es más pesado de aprender y de correr en un plan gratuito de 512 MB. *Se resigna:* la estructura impuesta de un framework más grande (la organización por módulos la definimos nosotros) y el tipado de Python con FastAPI, que genera la documentación de la API sola.

3. Base de datos — PostgreSQL en Neon (plan gratuito), con Prisma como ORM. *Alternativas:* MySQL en Aiven o Railway; MongoDB Atlas; Supabase. *Fundamento:* el negocio es relacional (usuarios, menús, pedidos y pagos con claves foráneas) y la regla de cupo necesita **transacciones con bloqueo de fila** (`SELECT … FOR UPDATE`), que PostgreSQL resuelve bien; MongoDB complicaría esa garantía. Neon es gratis, remoto y con TLS. Prisma da migraciones versionadas en el repositorio y consultas tipadas. *Se resigna:* el panel y la autenticación incluidos de Supabase, y algo de control fino del SQL (los bloqueos se escriben con `$queryRaw`). El plan gratuito tiene poco almacenamiento y la base se suspende sin uso (la primera consulta tarda un poco más).

4. Autenticación — propia, con bcrypt y JWT (8 h) y roles en la tabla `Rol`. *Alternativas:* Auth0 o Clerk (proveedor externo); sesiones con cookie guardadas en la base. *Fundamento:* son pocas reglas, se pueden probar con tests de integración (401 y 403) y se explican completas en la defensa. El JWT no necesita guardar sesiones; para poder cerrar sesión de verdad se agregó `versionToken`, que invalida los tokens viejos. *Se resigna:* inicio de sesión con Google y verificación del email que darían los proveedores externos; como el token vive en `localStorage`, depende de que el frontend no tenga XSS (React escapa el contenido).

5. Despliegue y CI/CD — Render (API y sitio estático) + GitHub Actions. *Alternativas:* Vercel (frontend) + Railway (API); un VPS con Docker. *Fundamento:* Render tiene plan gratuito para los dos componentes, HTTPS automático, variables de entorno como secretos y *deploy hooks*; GitHub Actions es gratis en el repositorio y permite que el despliegue dependa de que los tests y la cobertura (≥ 65 %) pasen. Railway ya no tiene plan gratuito permanente; un VPS cuesta dinero y exige administrar el servidor. *Se resigna:* la API se duerme tras 15 minutos sin uso (la primera petición tarda cerca de un minuto) y los logs gratuitos de Render guardan poco historial.

6. Tecnología avanzada — integración con Mercado Pago (Checkout Pro), más logs estructurados. *Alternativas:* observabilidad sola con una herramienta externa (Better Stack, Grafana Cloud); caché con Redis. *Fundamento:* resuelve un problema real del negocio: las reservas que no se pagan ocupan cupo; ahora una reserva sólo se confirma con el pago aprobado y se libera sola si vence. El estado se consulta siempre a Mercado Pago (no se confía en el navegador) y la falla o

demora del servicio responde 502/503 sin dejar reservas trabadas. Mercado Pago es el medio de pago más usado en Argentina, tiene cuentas de prueba gratuitas y se usó con `fetch` sin SDK. Una caché no tiene sentido con tan pocos datos. Los logs JSON (pino) en Render permiten ver qué pasó con un pedido en un momento dado. *Se resigna:* Mercado Pago cobra comisión por venta en producción; en local no llega el webhook (se confirma cuando el cliente vuelve), y sin una herramienta de logs externa el historial es corto.

Declaración de uso de inteligencia artificial (consigna 8)

Durante el desarrollo se usó **Claude (Anthropic)**, a través de Claude Code, para: generación de código del backend y del frontend, escritura de tests, interpretación de errores del pipeline y de Mercado Pago, redacción de la documentación y elaboración de los diagramas (DFD, casos de uso, DER y arquitectura, a partir del código del repositorio). Los integrantes definieron el producto y sus reglas de negocio (ver la propuesta), eligieron entre las alternativas de esta página, probaron el sistema funcionando y asumen la responsabilidad de comprender y explicar todo lo entregado.

Módulo 1 — Acceso

1. Propósito

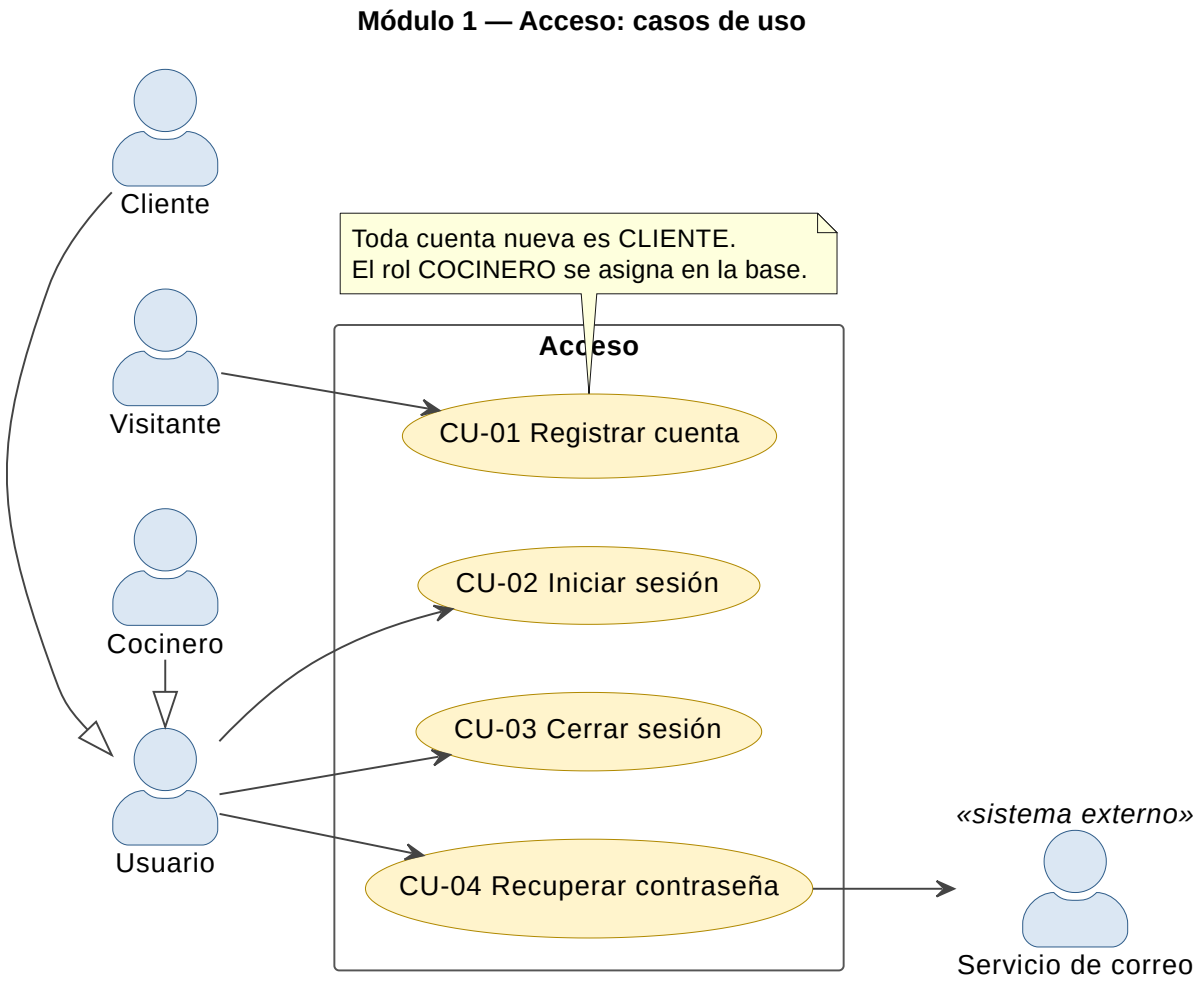
Identifica a cada persona que usa el sistema y decide qué puede hacer. Cubre el registro, el inicio y cierre de sesión, la validación del token en cada petición, los permisos por rol y la recuperación de la contraseña por correo. Corresponde al **proceso 1 “Gestionar acceso”** de los DFD y a la sección 3.4 de la consigna (autenticación y control de acceso).

Rol	Cómo se obtiene	Qué puede hacer
`CLIENTE`	Todo registro crea un cliente	Ver menús, reservar y pagar, ver y cancelar **sus** pedidos
`COCINERO`	Se asigna en la base (por ejemplo con `npx prisma studio`)	Crear, modificar y borrar menús; ver **todos** los pedidos

2. Dónde está en el código

Capa	Archivo	Qué hace
Base de datos	`backend/prisma/schema.prisma` (modelos `Rol`, `Usuario`, `CodigoRecuperacion`)	Tablas del módulo. La migración inicial inserta los roles
API	`backend/src/routes/auth.ts`	Endpoints de registro, login, logout, `/yo` y recuperación
API	`backend/src/middleware/auth.ts`	`autenticar` (401) y `exigirRol(...)` (403), usados por los demás módulos
API	`backend/src/lib/mailer.ts`	Envío de correos por SMTP (Nodemailer)
Frontend	`frontend/src/pantallas/Acceso.tsx`	Ingresar / crear cuenta
Frontend	`frontend/src/pantallas/Recuperar.tsx`	Los tres pasos de la recuperación
Frontend	`frontend/src/api/cliente.ts`	Guarda la sesión en `localStorage` y manda `Authorization: Bearer`
Frontend	`frontend/src/App.tsx`	Ante un 401 vuelve a la pantalla de ingreso; botón “Salir”
Tests	`backend/tests/integration/auth.test.ts`, `recuperacion.test.ts`, `tests/unit/mailer.test.ts`	11 + 7 tests de integración y 1 unitario

3. Casos de uso



Casos de uso del módulo Acceso

CU-01 Registrar cuenta

Campo	Detalle
Módulo	1 — Acceso
Actor principal	Visitante (persona sin cuenta)
Actores secundarios	—
Descripción	El visitante crea su cuenta y queda con la sesión iniciada como cliente.
Precondiciones	No existe un usuario con el email ingresado.
Disparador	El visitante elige “Crear cuenta” en la pantalla de ingreso.
Endpoint	`POST /api/auth/registro` → `201`

Flujo principal

1. El visitante completa nombre, email y contraseña y confirma.
2. El frontend envía los datos a `POST /api/auth/registro`.
3. La API valida los datos con zod: nombre no vacío, email válido (se pasa a minúsculas), contraseña de 8 a 72 caracteres.

4. La API verifica que el email no esté registrado.
5. La API calcula el hash de la contraseña con bcrypt (10 rondas, la sal va incluida en el hash).
6. La API guarda el usuario con el rol `CLIENTE` y responde `201` con id, nombre, email y rol.
7. El frontend inicia sesión automáticamente con los mismos datos (CU-02) y muestra el menú de la semana.

Flujos alternativos

- **3a. Datos inválidos:** la API responde `400` con la lista de campos y mensajes; el frontend muestra el mensaje y el visitante corrige.
- **4a. Email ya registrado:** la API responde `409 "Ya existe un usuario con ese email"`; no se crea nada.

Postcondiciones: existe un `Usuario` nuevo con rol `CLIENTE` y `passwordHash` (nunca la contraseña en texto plano). El visitante queda con sesión iniciada.

Reglas: nadie puede registrarse como cocinero desde la API: el rol se asigna en la base.

CU-02 Iniciar sesión

Campo	Detalle
Módulo	1 — Acceso
Actor principal	Usuario (Cliente o Cocinero)
Precondiciones	El usuario tiene cuenta.
Disparador	El usuario completa “Ingresar”.
Endpoint	`POST /api/auth/login` → `200 { token, usuario }`

Flujo principal

1. El usuario ingresa email y contraseña.
2. La API valida el formato (zod) y busca el usuario por email, con su rol.
3. La API compara la contraseña con el hash usando `bcrypt.compare`.
4. La API firma un JWT con `{ id, v }` (`v` = `versionToken` actual del usuario) que vence a las **8 horas**.
5. La API responde el token y los datos públicos del usuario (id, nombre, rol).
6. El frontend guarda la sesión en `localStorage` y muestra las pestañas que corresponden al rol.

Flujos alternativos

- **2a / 3a. Email inexistente o contraseña incorrecta:** la API responde `401 "Email o contraseña incorrectos"` (el mismo mensaje en los dos casos, para no revelar qué cuentas existen) y registra el evento `login_fallido` en el log.
- **2b. Datos con formato inválido:** `400`.

Postcondiciones: el usuario tiene un token válido; cada petición posterior lo manda en `Authorization: Bearer <token>`.

CU-03 Cerrar sesión

Campo	Detalle
Módulo	1 — Acceso

Campo	Detalle
Actor principal	Usuario (Cliente o Cocinero)
Precondiciones	El usuario tiene la sesión iniciada.
Disparador	El usuario toca “Salir”.
Endpoint	`POST /api/auth/logout` → `204`

Flujo principal

1. El frontend llama a `POST /api/auth/logout` con el token.
2. La API valida el token (`autenticar`).
3. La API incrementa `versionToken` del usuario: **todos** los tokens emitidos antes dejan de valer, en cualquier dispositivo.
4. La API responde `204`; el frontend borra la sesión guardada y vuelve a la pantalla de ingreso.

Flujos alternativos

- **2a. Token ya inválido o vencido:** la API responde `401`; el frontend igual borra la sesión local.

Postcondiciones: el token anterior responde `401 "La sesión ya no es válida"` si se vuelve a usar.

CU-04 Recuperar contraseña

Campo	Detalle
Módulo	1 — Acceso
Actor principal	Usuario (Cliente o Cocinero)
Actores secundarios	Servicio de correo (SMTP de Gmail)
Precondiciones	El usuario tiene acceso a la casilla de su email.
Disparador	El usuario toca “¿Olvidaste tu contraseña?”.
Endpoints	`POST /api/auth/recuperar`, `POST /api/auth/recuperar/verificar`, `POST /api/auth/recuperar/cambiar`

Flujo principal

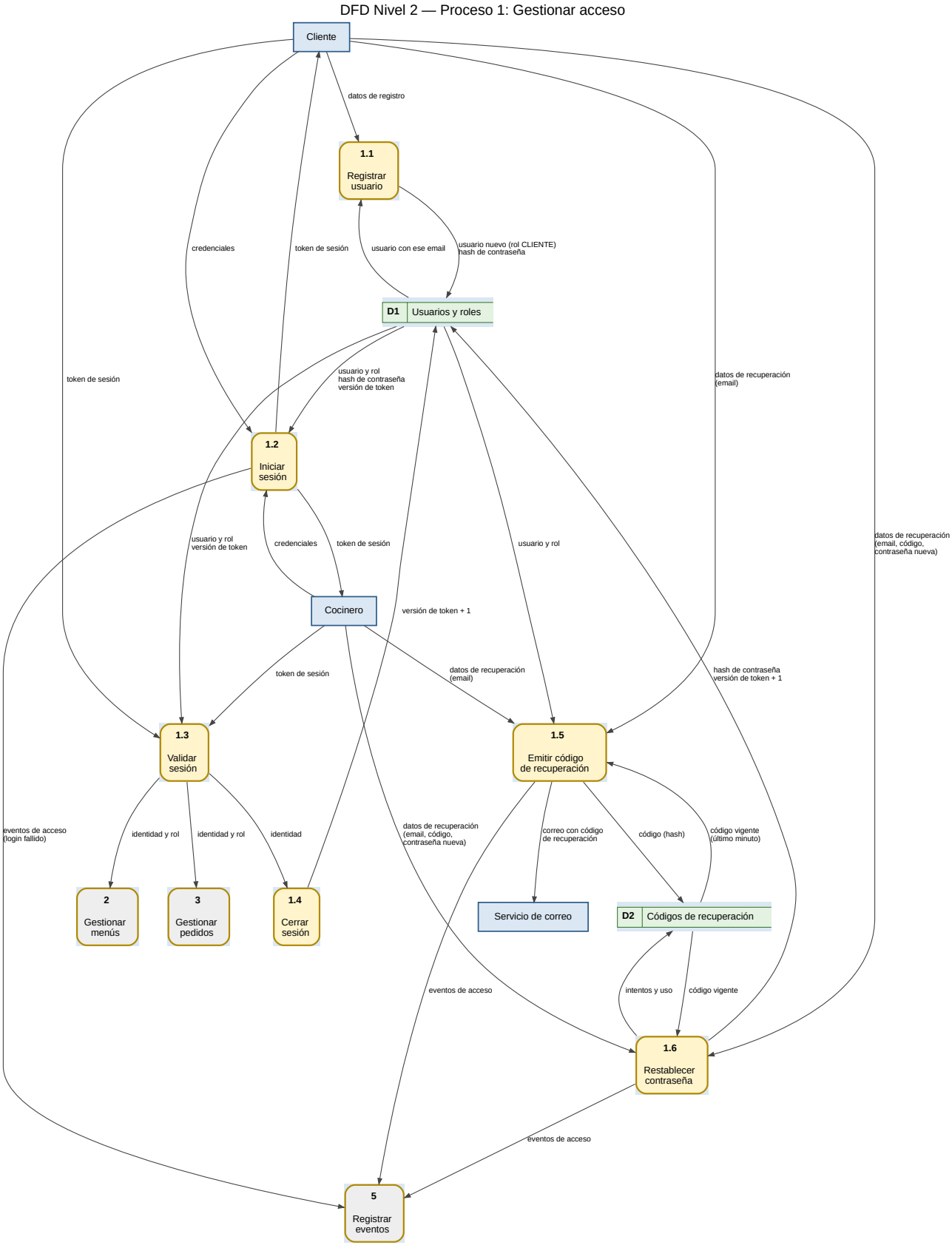
1. El usuario ingresa su email.
2. La API busca el usuario y verifica que no se haya pedido un código en el último minuto.
3. La API genera un código aleatorio de 6 dígitos (`crypto.randomInt`), invalida los códigos anteriores y guarda **sólo el HMAC** del código, con vencimiento a los 15 minutos.
4. La API envía el código por correo (Servicio de correo) y responde siempre el mismo mensaje: “Si el email está registrado, te enviamos un código”.
5. El usuario ingresa el código; la API lo verifica (`/verificar`).
6. El usuario elige la contraseña nueva (8 a 72 caracteres) y confirma (`/cambiar`).
7. La API vuelve a verificar el código y, en una transacción, lo marca como usado **sólo si nadie lo usó antes**, guarda el hash bcrypt de la contraseña nueva e incrementa `versionToken` (se cierran las sesiones abiertas).
8. La API registra `password_restablecida` y responde; el usuario ingresa con la contraseña nueva (CU-02).

Flujos alternativos

- **2a. Email no registrado:** no se envía nada, pero la respuesta es la misma (no revela quién tiene cuenta).
- **2b. Ya se pidió un código hace menos de un minuto:** no se envía otro; la respuesta es la misma.
- **4a. El servidor de correo no está configurado o falla:** `503` “No pudimos enviar el correo. Probá de nuevo en unos minutos” y evento `correo_fallido` / `correo_sin_configurar` en el log.
- **5a. Código incorrecto:** `400 CODIGO_INCORRECTO` y suma un intento. Al 5.º intento fallido el código se invalida: `400 CODIGO_BLOQUEADO`.
- **5b. Código vencido:** `400 CODIGO_VENCIDO`.
- **5c. Código ya usado o reemplazado por uno nuevo:** `400 CODIGO_USADO`.
- **7a. Dos pedidos simultáneos con el mismo código:** sólo uno logra marcarlo como usado; el otro recibe `400 CODIGO_USADO`.

Postcondiciones: la contraseña anterior deja de funcionar, el código no se puede volver a usar y los tokens anteriores dan `401`.

4. DFD Nivel 2 — Proceso 1 “Gestionar acceso”



DFD Nivel 2 del proceso 1

N.º	Proceso	Entrada	Salida	Almacenes
1.1	Registrar usuario	datos de registro	usuario nuevo	D1 (lee el email, escribe el usuario)
1.2	Iniciar sesión	credenciales	token de sesión; evento `login_fallido`	D1 (lee)
1.3	Validar sesión	token de sesión	identidad y rol → procesos 2, 3 y 1.4	D1 (lee `versionToken` y rol)
1.4	Cerrar sesión	identidad	—	D1 (escribe `versionToken + 1`)
1.5	Emitir código de recuperación	email	correo con código	D1 (lee), D2 (lee el último, escribe el nuevo)
1.6	Restablecer contraseña	email, código y contraseña nueva	—	D2 (intentos y uso), D1 (hash y `versionToken`)

El proceso 1.3 es el middleware `autenticar`: se ejecuta antes de **cada** endpoint protegido y es el que entrega la identidad a los procesos 2 y 3. El rol se lee **de la base**, nunca del cliente.

5. Datos que usa

Tabla	Campos clave	Notas
`Rol`	`id`, `nombre` (único)	Sólo dos filas: `COCINERO` y `CLIENTE` (las crea la migración)
`Usuario`	`email` (único), `passwordHash`, `rolId`, `versionToken`	`versionToken` sube en cada logout y al restablecer la contraseña
`CodigoRecuperacion`	`codigoHash`, `venceEn`, `usadoEn`, `intentos`	Se borra en cascada con el usuario

6. API

Método	Ruta	Protección	Respuestas
POST	`/api/auth/registro`	pública	201, 400, 409
POST	`/api/auth/login`	pública	200, 400, 401
POST	`/api/auth/logout`	token	204, 401
GET	`/api/auth/yo`	token	200 `{ id, rol }`, 401
POST	`/api/auth/recuperar`	pública	200 (siempre el mismo mensaje), 400, 503
POST	`/api/auth/recuperar/verificar`	pública	200, 400
POST	`/api/auth/recuperar/cambiar`	pública	200, 400

Cómo se protege cualquier endpoint de otro módulo:

```
menusRouter.use(autenticar);                          // 401 sin token, vencido o cerrado
menusRouter.post('/', exigirRol('COCINERO'), ...);    // 403 si el rol no corresponde
```

7. Reglas y validaciones (dónde está cada una)

Regla	Dónde
Email válido y en minúsculas, contraseña de 8 a 72 caracteres	`esquemaEmail` / `esquemaPassword` (zod) en `routes/auth.ts`
Contraseña guardada con hash y sal (bcrypt)	`routes/auth.ts`, registro y `/recuperar/cambiar`
Token sólo por encabezado `Authorization` (nunca en la URL)	`middleware/auth.ts`; el logger oculta ese encabezado (`redact`)
El id del usuario sale del token, no de un parámetro	`autenticar` → `req.usuario`
Token inválido tras logout	comparación `usuario.versionToken !== datos.v` en `autenticar`
Un código por minuto, 15 minutos de validez, 5 intentos, un solo uso	constantes al principio de la sección de recuperación en `routes/auth.ts`
La respuesta de `/recuperar` no revela si el email existe	`routes/auth.ts`

8. Pantallas

Pantalla	Qué muestra
Ingresar / Crear cuenta (`Acceso.tsx`)	Formulario con pestañas; tras registrarse entra solo
Recuperar contraseña (`Recuperar.tsx`)	Paso 1: email · Paso 2: código · Paso 3: contraseña nueva; botón para pedir otro código
Encabezado (`App.tsx`)	Nombre, rol (“Cocina” o “Cliente”) y botón “Salir”

Si cualquier petición devuelve `401`, el frontend borra la sesión y muestra “Tu sesión venció. Ingresá de nuevo.”

9. Tests que lo cubren

- `auth.test.ts`: registro correcto con la contraseña hasheada (201), datos inválidos (400), email duplicado (409), login correcto y con contraseña incorrecta (401), petición sin token, con token inválido y con token vencido (401), cliente en una ruta de cocinero (403), cocinero que pasa porque el rol se lee de la base, y token que deja de valer después del logout (401).
- `recuperacion.test.ts`: flujo completo, misma respuesta para un email inexistente, código incorrecto con bloqueo a los 5 intentos, código vencido, código nuevo que invalida el anterior y límite de uno por minuto, validación de la contraseña nueva (400) y correo que no se puede mandar (503).
- `mailer.test.ts`: el envío de correo sin SMTP configurado responde 503.

10. Para modificarlo

Si piden…	Tocar
Un campo nuevo del usuario (por ejemplo, teléfono)	`schema.prisma` + migración → `esquemaRegistro` en `routes/auth.ts` → formulario de `Acceso.tsx`
Cambiar la duración de la sesión	`expiresIn: '8h'` en `routes/auth.ts`
Un rol nuevo	fila en la tabla `Rol` (migración) y `exigirRol('NUEVO_ROL')` en las rutas

Si piden…	Tocar
Más exigencia en la contraseña	`esquemaPassword` en `routes/auth.ts` (el mensaje de error sale solo en el 400)

Módulo 2 — Menús

1. Propósito

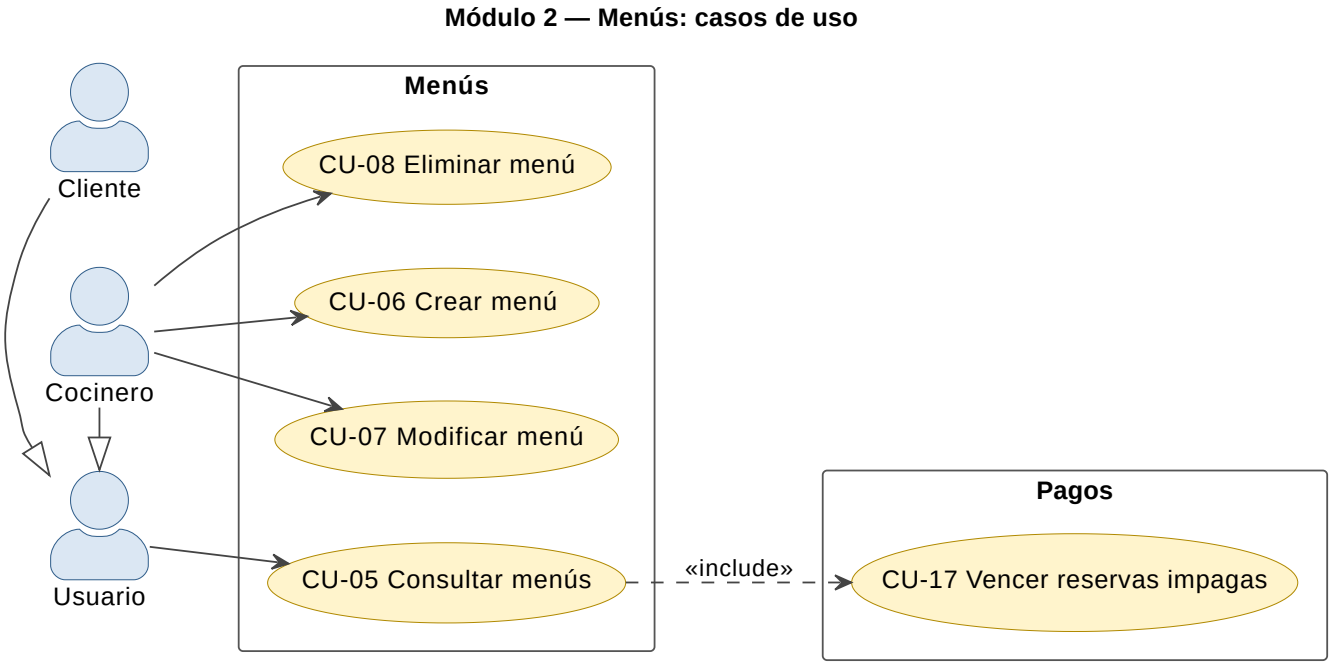
Permite que quien cocina publique el menú de cada día con su **cupo máximo**, su **horario de corte** y su **precio**, y que todos los usuarios vean los menús con cuántas viandas quedan. Corresponde al **proceso 2 “Gestionar menús”** de los DFD.

La API calcula y devuelve en cada menú `reservadas`, `disponibles` y `abierto` (si todavía se puede reservar). El frontend no calcula ninguna regla: sólo muestra lo que recibe (consigna 3.2).

2. Dónde está en el código

Capa	Archivo	Qué hace
Base de datos	`backend/prisma/schema.prisma` (modelo `MenuDia`)	Tabla de menús
API	`backend/src/routes/menus.ts`	CRUD de menús y cálculo de disponibilidad (`conTotales`)
Reglas	`backend/src/domain/reglasPedido.ts` (`yaPasoElCorte`, `instanteDeCorte`)	Si ya pasó la hora de corte (hora de Argentina, UTC-3)
Reglas	`backend/src/lib/pagos.ts` (`OCUPAN_CUPO`, `vencerReservasImpagas`)	Qué pedidos ocupan cupo y liberación de reservas vencidas
Frontend	`frontend/src/pantallas/Menus.tsx`	Tarjetas de menús, formulario del cocinero, botón “Reservar y pagar”
Tests	`backend/tests/integration/menus.test.ts`	10 tests de integración

3. Casos de uso



Casos de uso del módulo Menús

CU-05 Consultar menús

Campo	Detalle
Módulo	2 — Menús
Actor principal	Usuario (Cliente o Cocinero)
Precondiciones	El usuario tiene la sesión iniciada (CU-02).
Disparador	El usuario abre la pestaña “Menú de la semana” / “Menús de la semana”.
Relaciones	`<<include>>` CU-17 Vencer reservas impagas
Endpoints	`GET /api/menus`, `GET /api/menus/:id`

Flujo principal

1. El frontend pide `GET /api/menus` con el token.
2. La API valida la sesión (proceso 1.3).
3. La API cancela las reservas cuyo plazo para pagar ya venció (CU-17), para que no ocupen cupo.
4. La API lee los menús ordenados por fecha y, para cada uno, suma las viandas de pedidos `PENDIENTE_PAGO` y `CONFIRMADO`.
5. La API devuelve cada menú con `reservadas`, `disponibles = cupoMaximo − reservadas` (nunca negativo) y `abierto` (tiene precio y no pasó la hora de corte).
6. El frontend muestra una tarjeta por día con el precio, las viandas reservadas sobre el cupo y la hora de corte. Al cliente le agrega el botón “Reservar y pagar” si `abierto` es verdadero y quedan viandas; al cocinero, los botones para editar y borrar.

Flujos alternativos

- **2a. Sin sesión o sesión vencida:** `401`; el frontend vuelve a la pantalla de ingreso.
- **4a. Menú pedido por id que no existe:** `404 "Menú no encontrado"`.

- **6a. “Ver días anteriores”:** por defecto se ocultan los días pasados; el interruptor los muestra.

Postcondiciones: ninguna en los datos de menús; sólo se liberan reservas vencidas.

CU-06 Crear menú

Campo	Detalle
Módulo	2 — Menús
Actor principal	Cocinero
Precondiciones	Sesión iniciada con rol `COCINERO`.
Disparador	El cocinero toca “Nuevo menú”.
Endpoint	`POST /api/menus` → `201`

Flujo principal

1. El cocinero completa fecha, descripción, cupo máximo, precio y hora de corte.
2. La API verifica el rol (`exigirRol('COCINERO')`).
3. La API valida con zod: fecha `AAAA-MM-DD`, descripción no vacía, cupo entero ≥ 1, precio entero ≥ 1 (pesos, sin centavos) y hora `HH:mm` de 00:00 a 23:59.
4. La API verifica que no exista otro menú para esa fecha.
5. La API guarda el menú y responde `201` con el menú y sus totales (0 reservadas).
6. El frontend muestra “Menú creado” y lo agrega a la lista.

Flujos alternativos

- **2a. Usuario con rol CLIENTE:** `403 "No tenés permiso para esta acción"`.
- **3a. Datos inválidos:** `400` con el campo y el mensaje (por ejemplo “Formato HH:mm”).
- **4a. Ya existe un menú para esa fecha:** `409`.

Postcondiciones: existe un `MenuDia` nuevo y los clientes lo ven abierto hasta la hora de corte.

CU-07 Modificar menú

Campo	Detalle
Módulo	2 — Menús
Actor principal	Cocinero
Precondiciones	Sesión con rol `COCINERO`; el menú existe.
Disparador	El cocinero toca “Editar” en un menú.
Endpoint	`PUT /api/menus/:id` → `200`

Flujo principal

1. El frontend abre el formulario con los datos actuales; el cocinero los cambia y guarda.
2. La API verifica el rol y valida los datos igual que en CU-06.
3. La API verifica que el menú exista y que la nueva fecha no la use otro menú.
4. La API actualiza el menú y lo devuelve con sus totales recalculados.

Flujos alternativos

- **2a. Rol CLIENTE:** `403`. **2b. Datos inválidos:** `400`.
- **3a. El menú no existe:** `404`. **3b. La fecha ya la tiene otro menú:** `409`.

Postcondiciones: el menú queda con los datos nuevos. Si se baja el cupo por debajo de lo reservado, `disponibles` queda en 0 (nunca negativo) y no se aceptan más pedidos; las reservas existentes se mantienen.

CU-08 Eliminar menú

Campo	Detalle
Módulo	2 — Menús
Actor principal	Cocinero
Precondiciones	Sesión con rol `COCINERO`; el menú existe y **no tiene pedidos**.
Disparador	El cocinero toca “Borrar” y confirma.
Endpoint	`DELETE /api/menus/:id` → `204`

Flujo principal

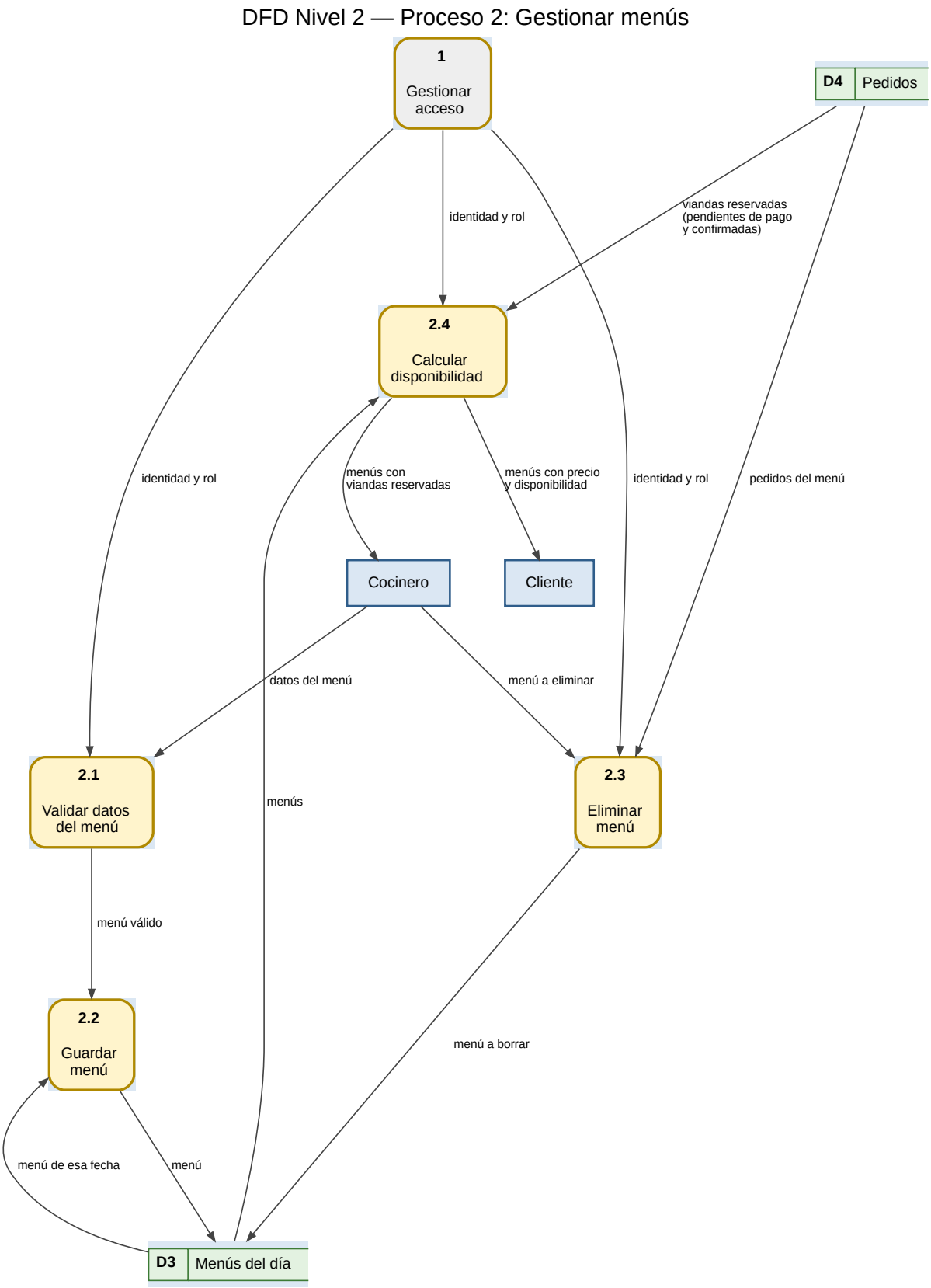
1. El frontend pide confirmación (“¿Borrar el menú del …?”).
2. La API verifica el rol y que el menú exista.
3. La API cuenta los pedidos del menú (de cualquier estado); si no hay, lo borra y responde `204`.

Flujos alternativos

- **1a. El cocinero no confirma:** no se envía nada.
- **2a. Rol CLIENTE:** `403`. **2b. No existe:** `404`.
- **3a. El menú tiene pedidos:** `409 "No se puede borrar un menú que tiene pedidos"` (además, la clave foránea es `RESTRICT`).

Postcondiciones: el menú deja de existir.

4. DFD Nivel 2 — Proceso 2 “Gestionar menús”



DFD Nivel 2 del proceso 2

N.º	Proceso	Entrada	Salida	Almacenes
2.1	Validar datos del menú	datos del menú, identidad y rol	menú válido	—
2.2	Guardar menú	menú válido	—	D3 (lee la fecha, escribe el menú)
2.3	Eliminar menú	menú a eliminar, identidad y rol	—	D4 (lee sus pedidos), D3 (borra)
2.4	Calcular disponibilidad	identidad y rol	menús con precio y disponibilidad (cliente), menús con viandas reservadas (cocinero)	D3 (lee), D4 (suma viandas)

5. Datos que usa

Tabla	Campos	Notas
`MenuDia`	`fecha` (DATE, única), `descripcion`, `cupoMaximo`, `horaCorte` (“HH:mm”), `precio` (pesos)	Un menú por día
`Pedido` (lectura)	`menuId`, `cantidad`, `estado`	Ocupan cupo los estados `PENDIENTE_PAGO` y `CONFIRMADO`

6. API

Método	Ruta	Rol	Respuestas
GET	`/api/menus`	cualquiera con sesión	200, 401
GET	`/api/menus/:id`	cualquiera con sesión	200, 400, 401, 404
POST	`/api/menus`	COCINERO	201, 400, 401, 403, 409
PUT	`/api/menus/:id`	COCINERO	200, 400, 401, 403, 404, 409
DELETE	`/api/menus/:id`	COCINERO	204, 400, 401, 403, 404, 409

Ejemplo de respuesta de `GET /api/menus`:

```
[{ "id": 3, "fecha": "2026-10-15T00:00:00.000Z", "descripcion": "Milanesa con puré",
   "cupoMaximo": 20, "horaCorte": "10:00", "precio": 6500,
   "reservadas": 12, "disponibles": 8, "abierto": true }]
```

7. Reglas y validaciones

Regla	Dónde
Formato de fecha, hora de corte, cupo y precio	`esquemaMenu` (zod) en `routes/menus.ts`
Un menú por fecha	índice único `MenuDia_fecha_key` + control previo (409) en `routes/menus.ts`
La hora de corte es hora de Argentina (UTC-3, sin horario de verano)	`instanteDeCorte` en `domain/reglasPedido.ts`

Regla	Dónde
Un menú sin precio (precio 0, menús anteriores a los pagos) no se puede reservar	`abierto` en `conTotales` y control `SIN_PRECIO` en `routes/pedidos.ts`
No se borra un menú con pedidos	`routes/menus.ts` (409) y FK `ON DELETE RESTRICT`

8. Pantallas

Pantalla	Cliente	Cocinero
Menú de la semana (`Menus.tsx`)	Tarjeta por día con etiqueta “Quedan N”, “Agotado” o “Cerrado”, descripción, precio, barra de cupo, hora de corte, cantidad y “Reservar y pagar” (sólo si está abierto y quedan viandas)	La misma tarjeta, con “Editar” y “Borrar”; botón “Nuevo menú” con formulario

9. Tests que lo cubren (`menus.test.ts`)

Crear (201), listar y ver uno, modificar, eliminar, no borrar con pedidos (409), no repetir fecha (409), datos inválidos (400), menú inexistente (404), sin token (401) y cliente que intenta crear (403).

10. Para modificarlo

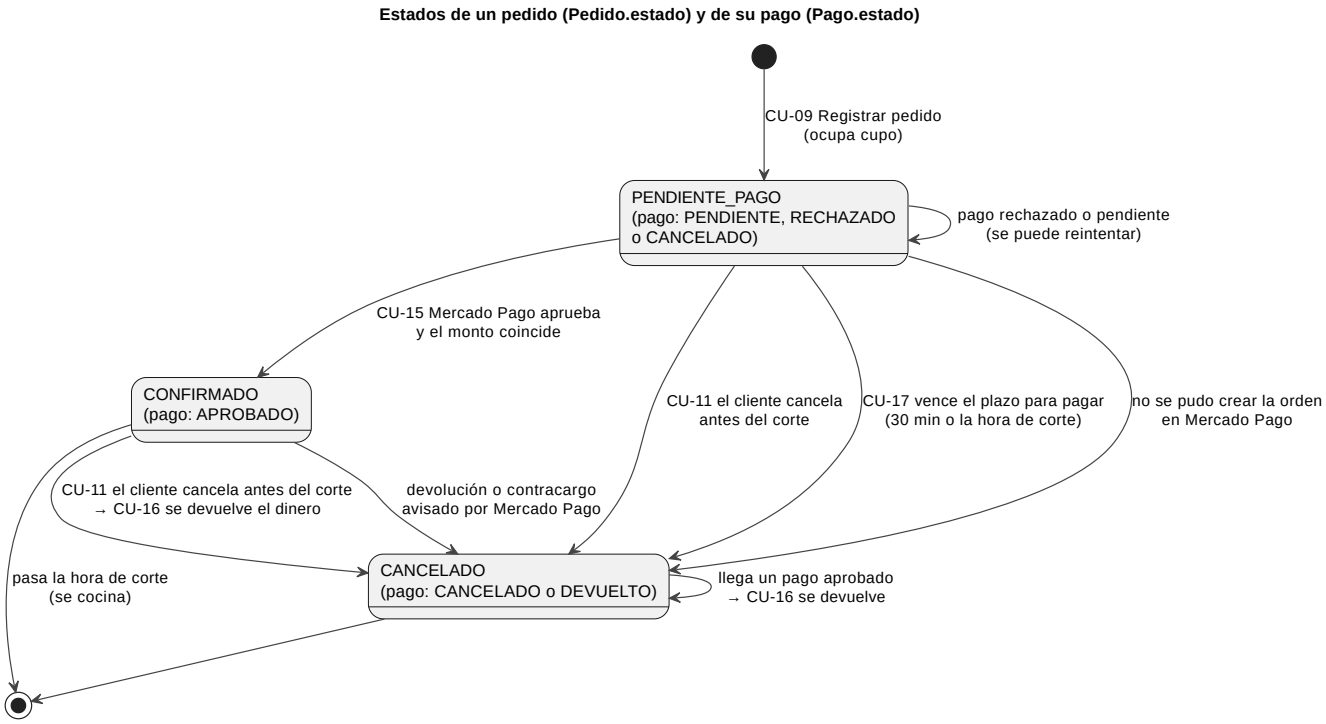
Si piden…	Tocar
Un campo nuevo del menú (por ejemplo “es vegetariano”)	`schema.prisma` + `npx prisma migrate dev` → `esquemaMenu` en `routes/menus.ts` → `tipos.ts` y formulario/tarjeta en `Menus.tsx`
Filtrar menús por fecha	parámetros de consulta en `menusRouter.get('/')` (`where` de `findMany`)
Un total (por ejemplo viandas reservadas de la semana)	nuevo endpoint en `routes/menus.ts` con `prisma.pedido.aggregate` sobre `OCUPAN_CUPO`

Módulo 3 — Pedidos

1. Propósito

Es el corazón del negocio: el cliente reserva viandas de un menú, ve sus reservas y las cancela; quien cocina ve cuántas viandas tiene que preparar cada día. Aplica las reglas de la propuesta en el servidor: **cupo máximo**, **horario de corte para pedir** y **para cancelar**, y **permisos por rol**. Corresponde al **proceso 3 “Gestionar pedidos”** de los DFD.

Un pedido nace `PENDIENTE_PAGO` y ocupa su lugar en el cupo mientras se paga (ver el módulo 4).



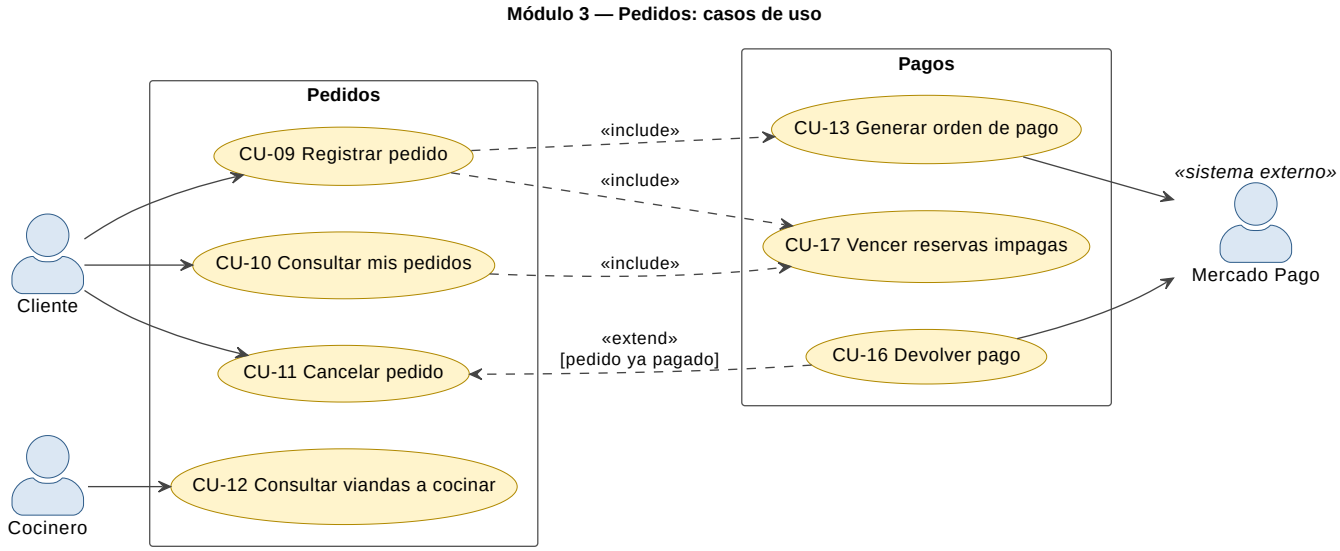
Estados del pedido y del pago

2. Dónde está en el código

Capa	Archivo	Qué hace
Base de datos	`backend/prisma/schema.prisma` (modelo `Pedido`, enum `EstadoPedido`)	Tabla de pedidos
API	`backend/src/routes/pedidos.ts`	Crear (con transacción y bloqueo), listar, pagar y cancelar
Reglas (funciones puras)	`backend/src/domain/reglasPedido.ts`	`validarNuevoPedido`, `validarCancelacion`, `cuposDisponibles`, `vencimientoDelPago`
Pagos	`backend/src/lib/mercadoPago.ts`, `backend/src/lib/pagos.ts`	Orden de pago, devoluciones y vencimiento (módulo 4)
Frontend	`frontend/src/pantallas/Menus.tsx` (`reservar`)	Botón “Reservar y pagar”

Capa	Archivo	Qué hace
Frontend	`frontend/src/pantallas/Pedidos.tsx`	“Mis pedidos” (cliente) y “Pedidos a cocinar” (cocinero)
Tests	`backend/tests/unit/reglasPedido.test.ts`, `backend/tests/integration/pedidos.test.ts`	19 unitarios y 11 de integración

3. Casos de uso



Casos de uso del módulo Pedidos

CU-09 Registrar pedido ★ (caso de uso principal)

Campo	Detalle
Módulo	3 — Pedidos
Actor principal	Cliente
Actores secundarios	Mercado Pago (a través de CU-13)
Descripción	El cliente reserva una o más viandas de un menú; la reserva queda pendiente de pago y el cliente pasa a pagar en Mercado Pago.
Precondiciones	Sesión iniciada con rol `CLIENTE`. El menú existe, tiene precio y está abierto.
Disparador	El cliente elige la cantidad y toca “Reservar y pagar” en la tarjeta de un menú.
Relaciones	`<<include>>` CU-17 Vencer reservas impagas · `<<include>>` CU-13 Generar orden de pago
Endpoint	`POST /api/pedidos { menuId, cantidad }` → `201 { ...pedido, pago, urlPago }`

Flujo principal

1. El cliente indica la cantidad y toca “Reservar y pagar”.
2. El frontend envía `POST /api/pedidos` con el token.
3. La API valida la sesión y que el rol sea `CLIENTE`.

4. La API valida los datos con zod (`menuId` entero positivo, `cantidad` entero ≥ 1).
5. La API abre una transacción y **bloquea la fila del menú** (`SELECT … FOR UPDATE`): si llegan dos pedidos a la vez, el segundo espera al primero.
6. La API verifica que el menú tenga precio.
7. La API cancela las reservas impagas vencidas (CU-17) para que no ocupen cupo.
8. La API suma las viandas de los pedidos `PENDIENTE_PAGO` y `CONFIRMADO` del menú.
9. La API aplica `validarNuevoPedido`: la hora de corte no pasó y la cantidad no supera las viandas disponibles.
10. La API crea el pedido (`PENDIENTE_PAGO`) y su pago (`PENDIENTE`, `monto = cantidad × precio`, vencimiento = 30 minutos o la hora de corte, lo que ocurra antes) y confirma la transacción.
11. La API genera la orden de pago en Mercado Pago (CU-13), guarda su id y la URL de pago.
12. La API responde `201` con el pedido, el pago y `urlPago`.
13. El frontend avisa “Te llevamos a Mercado Pago para pagar tu reserva…” y redirige al cliente a `urlPago` (continúa en CU-14).

Flujos alternativos

- **3a. Sin sesión:** `401`. **3b. Rol COCINERO:** `403`.
- **4a. Datos inválidos:** `400` con el detalle por campo.
- **5a. El menú no existe:** `404 "Menú no encontrado"`; se deshace la transacción.
- **6a. Menú sin precio:** `409 SIN_PRECIO "Este menú todavía no tiene precio"`.
- **9a. Pasó la hora de corte:** `409 FUERA_DE_HORARIO`; se registra el evento `regla_negocio` en el log.
- **9b. No alcanza el cupo:** `409 CUPO_COMPLETO` con “No quedan viandas disponibles para este día” o “Sólo quedan N viandas”; se registra `regla_negocio`.
- **11a. Mercado Pago no responde, falla o no está configurado:** el pedido y su pago pasan a `CANCELADO` (libera el cupo) y la API responde `502`/`503` con “Probá de nuevo en unos minutos”; el error queda en el log.

Postcondiciones

- Éxito: existe un `Pedido` `PENDIENTE_PAGO` con su `Pago` `PENDIENTE` y la URL de pago; las viandas quedan apartadas del cupo hasta que se pague, se cancele o venza.
- Fracaso: no queda ninguna reserva ocupando cupo.

Reglas de negocio: cupo máximo por día; no se pide después de la hora de corte; dos pedidos simultáneos nunca superan el cupo (bloqueo de fila + transacción).

CU-10 Consultar mis pedidos

Campo	Detalle
Módulo	3 — Pedidos
Actor principal	Cliente
Precondiciones	Sesión iniciada con rol `CLIENTE`.
Disparador	El cliente abre la pestaña “Mis pedidos” (o vuelve de Mercado Pago).
Relaciones	`<<include>>` CU-17 Vencer reservas impagas
Endpoint	`GET /api/pedidos` → `200`

Flujo principal

1. El frontend pide `GET /api/pedidos`.
2. La API valida la sesión y cancela las reservas vencidas (CU-17).

3. La API lee **sólo los pedidos del usuario del token**, con su menú y el estado y monto de su pago, del más nuevo al más viejo.
4. La API agrega a cada pedido `cancelable` (no está cancelado y no pasó la hora de corte) y `pagable` (está `PENDIENTE_PAGO`).
5. El frontend agrupa los pedidos por día y muestra cada uno con su estado (“Pagado”, “Falta pagar”, “Pago rechazado”, “Cancelado”…) y los botones “Pagar $…” y “Cancelar” según `pagable` y `cancelable`.

Flujos alternativos

- **2a. Sin sesión:** `401`.
- **5a. Sin pedidos:** “No tenés pedidos. Reservá desde el menú de la semana.”
- **5b. “Ver días anteriores”:** muestra también los pedidos de días pasados.

Postcondiciones: ninguna (sólo se liberan reservas vencidas).

CU-11 Cancelar pedido ★ (caso de uso principal)

Campo	Detalle
Módulo	3 — Pedidos
Actor principal	Cliente
Actores secundarios	Mercado Pago (si hay que devolver el dinero)
Descripción	El cliente cancela una reserva propia antes de la hora de corte; si ya estaba pagada, se le devuelve el dinero.
Precondiciones	Sesión con rol `CLIENTE`; el pedido es del cliente y no está cancelado.
Disparador	El cliente toca “Cancelar” en “Mis pedidos” y confirma.
Relaciones	Extendido por CU-16 Devolver pago `[pedido ya pagado]`
Endpoint	`PATCH /api/pedidos/:id/cancelar` → `200`

Flujo principal

1. El frontend pide confirmación; si el pedido está pagado agrega “Te devolvemos el pago por Mercado Pago”.
2. El frontend envía `PATCH /api/pedidos/:id/cancelar`.
3. La API valida la sesión, el rol `CLIENTE` y que el id sea un entero positivo.
4. La API abre una transacción y **bloquea la fila del pedido** (`FOR UPDATE`) para que no se confirme un pago mientras se cancela.
5. La API verifica que el pedido exista y sea del usuario del token.
6. La API aplica `validarCancelacion`: el pedido no está cancelado y no pasó la hora de corte del menú.
7. Si el pago estaba `PENDIENTE` o `RECHAZADO`, lo pasa a `CANCELADO`.
8. La API pasa el pedido a `CANCELADO` (libera el cupo) y lo devuelve.
9. El frontend muestra “Pedido cancelado” y recarga la lista.

Flujos alternativos

- **3a. Rol COCINERO:** `403`. **3b. Id inválido:** `400`.
- **5a. El pedido no existe o es de otro cliente:** `404 "Pedido no encontrado"` (no revela que existe).

- **6a. Ya estaba cancelado:** `409 YA_CANCELADO`.
- **6b. Pasó la hora de corte:** `409 FUERA_DE_HORARIO` (“No se puede cancelar un pedido después del horario de corte”).
- **7a. El pedido estaba pagado (`APROBADO`):** se ejecuta CU-16 Devolver pago; el pago queda `DEVUELTO` y se registra `pago_devuelto`.
- **7b. Mercado Pago falla al devolver:** la transacción se deshace, el pedido **no** se cancela y la API responde `502`.

Postcondiciones: el pedido queda `CANCELADO`, sus viandas vuelven al cupo y, si estaba pagado, el dinero se devolvió.

CU-12 Consultar viandas a cocinar

Campo	Detalle
Módulo	3 — Pedidos
Actor principal	Cocinero
Precondiciones	Sesión con rol `COCINERO`.
Disparador	El cocinero abre la pestaña “Pedidos a cocinar”.
Endpoint	`GET /api/pedidos` → `200` (todos los pedidos)

Flujo principal

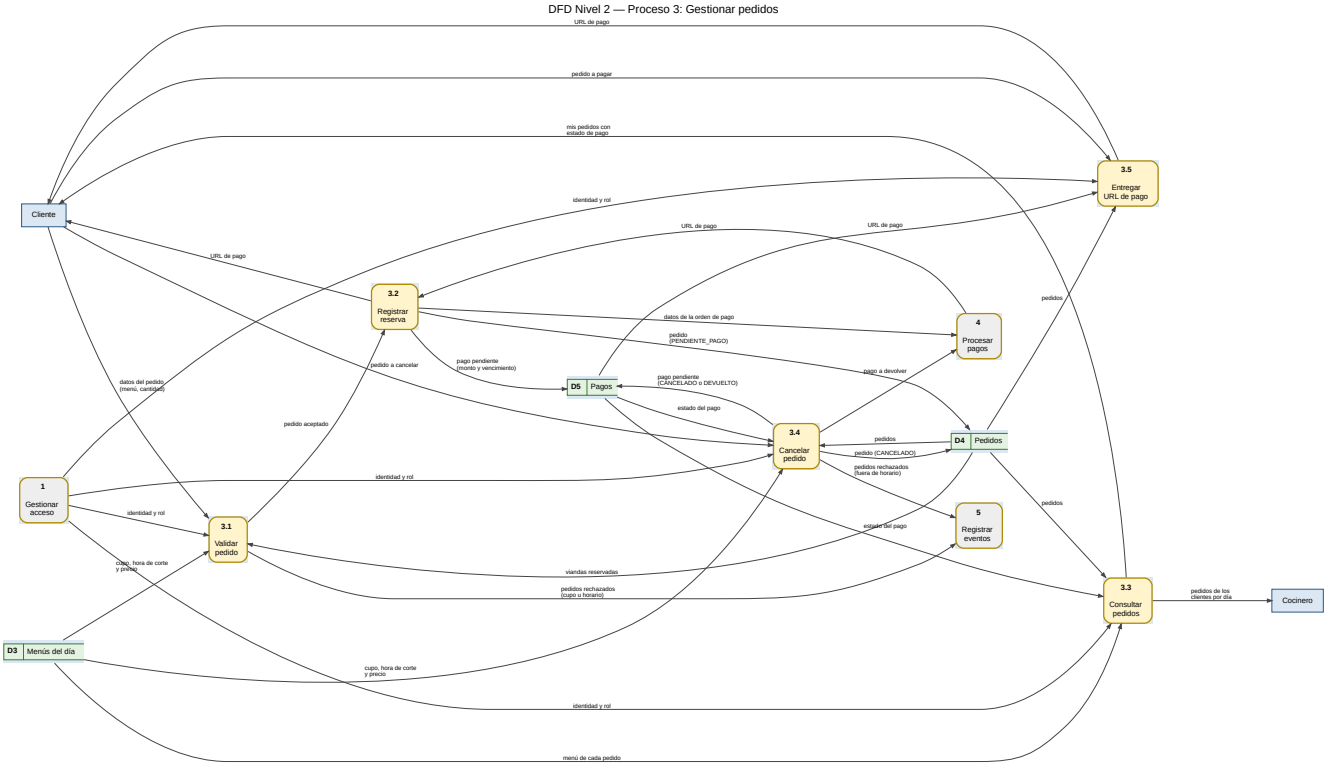
1. El frontend pide `GET /api/pedidos`.
2. La API valida la sesión y, por ser `COCINERO`, devuelve **todos** los pedidos con el nombre del cliente.
3. El frontend agrupa por día, muestra sólo los pedidos `CONFIRMADO` (pagados), con la cantidad y el nombre del cliente, y el total de viandas a cocinar de ese día.

Flujos alternativos

- **3a. Un día sin pedidos pagados:** “No hay pedidos pagados para este día.”
- **3b. “Ver días anteriores”:** incluye días pasados.

Postcondiciones: ninguna.

4. DFD Nivel 2 — Proceso 3 “Gestionar pedidos”



DFD Nivel 2 del proceso 3

N.º	Proceso	Entrada	Salida	Almacenes
3.1	Validar pedido	datos del pedido, identidad y rol	pedido aceptado; pedidos rechazados → 5	D3 (cupo, corte, precio), D4 (viandas reservadas)
3.2	Registrar reserva	pedido aceptado; URL de pago (de 4)	datos de la orden de pago → 4; URL de pago → Cliente	D4 (pedido), D5 (pago pendiente)
3.3	Consultar pedidos	identidad y rol	mis pedidos con estado de pago; pedidos de los clientes por día	D3, D4, D5 (lee)
3.4	Cancelar pedido	pedido a cancelar, identidad y rol	pago a devolver → 4; rechazos → 5	D3, D4, D5 (lee y escribe)
3.5	Entregar URL de pago	pedido a pagar, identidad y rol	URL de pago	D4, D5 (lee)

El flujo “pedido a cancelar o pagar” del Nivel 1 se divide aquí en “pedido a cancelar” (3.4) y “pedido a pagar” (3.5).

5. Datos que usa

Tabla	Campos	Notas
`Pedido`	`cantidad`, `estado`, `usuarioId`, `menuId`, `creadoEn`	`estado`: `PENDIENTE_PAGO` → `CONFIRMADO` o `CANCELADO`

Tabla	Campos	Notas
`MenuDia` (lectura y bloqueo)	`cupoMaximo`, `horaCorte`, `precio`, `fecha`	El `FOR UPDATE` se hace sobre esta fila
`Pago`	`monto`, `estado`, `venceEn`, `urlPago`	Uno por pedido (módulo 4)

6. API

Método	Ruta	Rol	Respuestas
GET	`/api/pedidos`	CLIENTE (los suyos) / COCINERO (todos)	200, 401
POST	`/api/pedidos`	CLIENTE	201, 400, 401, 403, 404, 409, 502, 503
POST	`/api/pedidos/:id/pagar`	CLIENTE	200 `{ urlPago }`, 401, 403, 404, 409 (módulo 4, CU-14)
PATCH	`/api/pedidos/:id/cancelar`	CLIENTE	200, 400, 401, 403, 404, 409, 502

Códigos de negocio que acompañan al 409: `CUPO_COMPLETO`, `FUERA_DE_HORARIO`, `SIN_PRECIO`, `YA_CANCELADO`, `NO_PAGABLE`, `CANTIDAD_INVALIDA`.

7. Reglas y validaciones

Regla	Dónde	Por qué ahí
Formato de `menuId` y `cantidad`	`esquemaPedido` (zod) en `routes/pedidos.ts`	Datos de entrada → 400
Cupo máximo y hora de corte para pedir	`validarNuevoPedido` en `domain/reglasPedido.ts`	Función pura: se prueba con tests unitarios sin base
Hora de corte para cancelar	`validarCancelacion` en `domain/reglasPedido.ts`	Ídem
Dos pedidos simultáneos no superan el cupo	transacción + `SELECT … FOR UPDATE` en `routes/pedidos.ts`	Sólo la base puede garantizarlo
El cliente sólo ve y cancela lo suyo	`where: { usuarioId }` y control `usuarioId !== req.usuario.id`	El id sale del token
Sólo el cliente reserva; sólo el cocinero ve todo	`exigirRol('CLIENTE')` y filtro por rol en `GET /`	Control en cada endpoint
Botones “Pagar” y “Cancelar”	`pagable` y `cancelable` calculados en la API	El frontend no calcula reglas

8. Pantallas

Pantalla	Cliente	Cocinero
Menú de la semana (`Menus.tsx`)	Cantidad + “Reservar y pagar” → redirige a Mercado Pago	—
Pedidos (`Pedidos.tsx`)	“Mis pedidos”: por día, estado del pago, “Pagar $…” y “Cancelar”	“Pedidos a cocinar”: por día, pedidos pagados con nombre del cliente y total de viandas a cocinar

9. Tests que lo cubren

- **Unitarios** (`reglasPedido.test.ts`, 19): conversión de la hora de corte a UTC, un minuto antes / justo en el corte, cupo disponible nunca negativo, pedido que completa exactamente el cupo, que lo supera, cupo completo, después del corte, cantidades inválidas, código 409, cancelación antes y después del corte y ya cancelada, 30 minutos para pagar sin pasar el corte, traducción de estados de Mercado Pago.
- **Integración** (`pedidos.test.ts`, 11): pedido pendiente de pago (201), supera el cupo (409 `CUPO_COMPLETO`), después del corte (409 `FUERA_DE_HORARIO`), **pedidos simultáneos que nunca superan el cupo**, datos inválidos (400) y menú inexistente (404), cocinero que intenta pedir (403), cada cliente ve sólo lo suyo y el cocinero todo, total a preparar, cancelación que libera el cupo, doble cancelación (409) y pedido ajeno (404), cancelación después del corte (409).

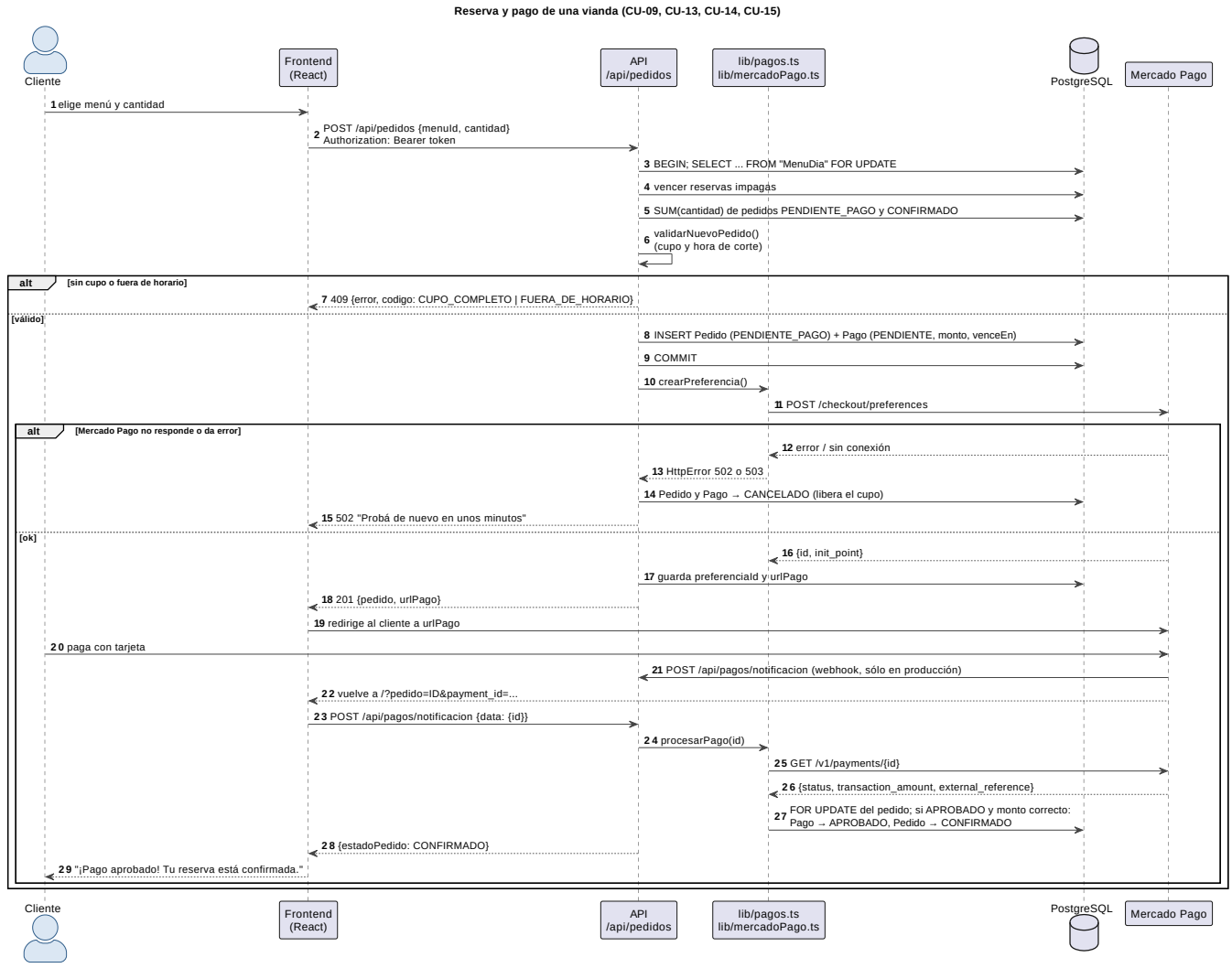
10. Para modificarlo

Si piden…	Tocar
Un máximo de viandas por pedido	nueva condición en `validarNuevoPedido` (con su test unitario) o `.max(n)` en `esquemaPedido`
Un comentario en el pedido (“sin sal”)	`schema.prisma` (campo en `Pedido`) + migración → `esquemaPedido` y `create` en `routes/pedidos.ts` → formulario en `Menus.tsx` y lista en `Pedidos.tsx`
Filtrar pedidos por fecha o estado	parámetros de consulta en `pedidosRouter.get('/')` (`where` de `findMany`)
Un endpoint con el total de viandas de un día	nueva ruta en `routes/pedidos.ts` con `prisma.pedido.aggregate({ _sum: { cantidad: true }, where: { estado: 'CONFIRMADO', menu: { fecha } } })` y `exigirRol('COCINERO')`

Módulo 4 — Pagos

1. Propósito

Cobra el **100 %** de cada reserva con Mercado Pago (Checkout Pro) y mantiene el estado de la reserva de acuerdo con el estado **real** del pago, que siempre se consulta a la API de Mercado Pago y nunca se toma de lo que mande el navegador. También devuelve el dinero cuando corresponde y libera el cupo de las reservas que no se pagan a tiempo. Corresponde al **proceso 4 “Procesar pagos”** de los DFD y es la **tecnología avanzada** “integración con un servicio de terceros” de la consigna (sección 3.5), incluido el comportamiento ante la falla o demora del servicio.



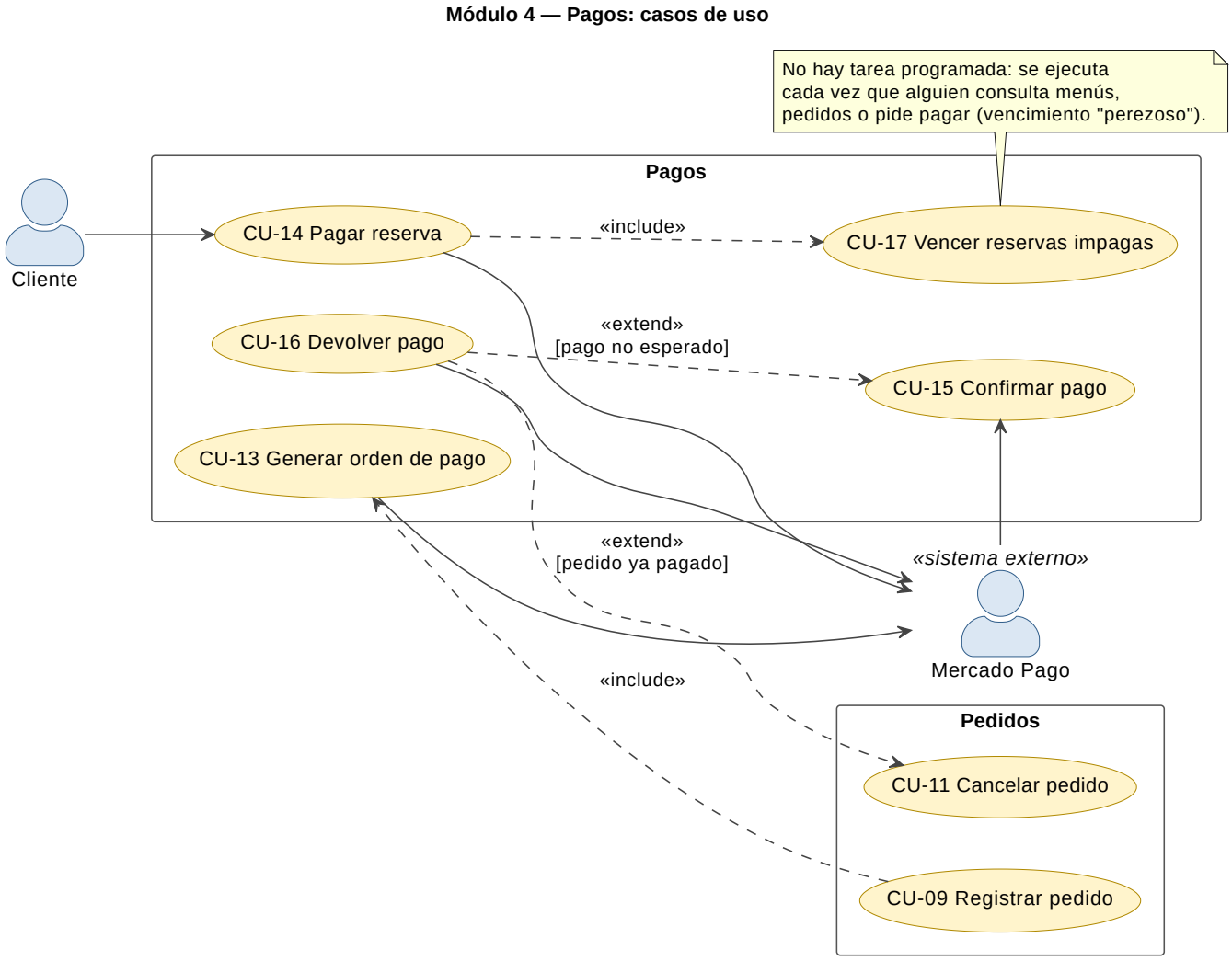
Secuencia de una reserva con su pago

2. Dónde está en el código

Capa	Archivo	Qué hace
Base de datos	`backend/prisma/schema.prisma` (modelo `Pago`, enum `EstadoPago`)	Un pago por pedido
Cliente de Mercado Pago	`backend/src/lib/mercadoPago.ts`	`crearPreferencia`, `obtenerPago`, `devolverPago` con `fetch` nativo (sin SDK)

Capa	Archivo	Qué hace
Lógica de pagos	`backend/src/lib/pagos.ts`	`procesarPago` (confirma o devuelve), `vencerReservasImpagas`, `OCUPAN_CUPO`
Reglas (puras)	`backend/src/domain/reglasPedido.ts`	`vencimientoDelPago`, `estadoDelPago`
API	`backend/src/routes/pagos.ts`	Webhook `POST /api/pagos/notificacion`
API	`backend/src/routes/pedidos.ts`	Crea la orden al reservar, `POST /:id/pagar`, devolución al cancelar
Frontend	`frontend/src/App.tsx` (`volverDeMercadoPago`)	Al volver de Mercado Pago pide a la API que consulte el pago
Frontend	`frontend/src/pantallas/Pedidos.tsx` (`pagar`)	Botón “Pagar $…” para reintentar
Tests	`backend/tests/integration/pagos.test.ts`	14 tests de integración (Mercado Pago simulado)

3. Casos de uso



Casos de uso del módulo Pagos

CU-13 Generar orden de pago

Campo	Detalle
Módulo	4 — Pagos
Actor principal	(incluido en CU-09 Registrar pedido)
Actores secundarios	Mercado Pago
Precondiciones	El pedido y su pago `PENDIENTE` ya se guardaron (CU-09, paso 10).
Endpoint externo	`POST` `https://api.mercadopago.com/checkout/preferences`

Flujo principal

1. La API arma la preferencia: un ítem con título “Vianda AAAA-MM-DD: descripción”, cantidad y precio unitario en ARS; `external_reference` = id del pedido.
2. Agrega las URL de vuelta (`FRONTEND_URL/?pedido=ID`), la URL de notificación (`API_URL/api/pagos/notificacion` , sólo si hay URL pública), el vencimiento (`expiration_date_to` = vencimiento del pago) y excluye pagos en efectivo y cajero (se acreditan tarde).

3. La envía a Mercado Pago con el `MP_ACCESS_TOKEN` en el encabezado `Authorization`.
4. Mercado Pago responde el id de la preferencia y su `init_point` (URL de pago).
5. La API guarda `preferenciaId` y `urlPago` en el `Pago`.

Flujos alternativos

- **3a. Falta `MP_ACCESS_TOKEN`:** `503 "Los pagos no están disponibles en este momento"` y evento `mercadopago_sin_configurar`.
- **3b. Sin conexión con Mercado Pago:** `502` y evento `mercadopago_sin_conexion`.
- **4a. Mercado Pago responde con error:** `502`; el detalle va sólo al log (`mercadopago_error`).
- En todos los casos alternativos CU-09 cancela el pedido para liberar el cupo.

Postcondiciones: el pago tiene URL de pago y vence en Mercado Pago a la misma hora que en el sistema.

CU-14 Pagar reserva

Campo	Detalle
Módulo	4 — Pagos
Actor principal	Cliente
Actores secundarios	Mercado Pago
Precondiciones	El cliente tiene una reserva `PENDIENTE_PAGO` sin vencer.
Disparador	Redirección automática después de CU-09, o botón “Pagar $…” en “Mis pedidos”.
Relaciones	`<<include>>` CU-17 Vencer reservas impagas
Endpoint	`POST /api/pedidos/:id/pagar` → `200 { urlPago }`

Flujo principal

1. El cliente toca “Pagar $…” en una reserva pendiente.
2. La API valida la sesión y el rol `CLIENTE`, y cancela las reservas vencidas (CU-17).
3. La API verifica que el pedido sea del cliente, siga `PENDIENTE_PAGO` y tenga URL de pago.
4. La API devuelve la URL de pago; el frontend redirige al cliente a Mercado Pago.
5. El cliente paga en la página de Mercado Pago (tarjeta o dinero en cuenta).
6. Mercado Pago avisa el pago a la API y devuelve al cliente al frontend; continúa en CU-15.

Flujos alternativos

- **3a. El pedido no existe o es de otro cliente:** `404`.
- **3b. Ya está pagado:** `409 NO_PAGABLE "Este pedido ya está pagado"`.
- **3c. Venció o se canceló:** `409 NO_PAGABLE "Esta reserva venció o se canceló. Hacé una nueva"`.
- **5a. El cliente abandona el pago:** al volver sin `payment_id`, el frontend avisa que puede pagar desde “Mis pedidos” antes de que venza.
- **5b. El pago es rechazado:** la reserva sigue pendiente y el cliente puede reintentar (vuelve al paso 1).

Postcondiciones: Mercado Pago tiene un pago asociado a la preferencia del pedido.

CU-15 Confirmar pago ★ (caso de uso principal)

Campo	Detalle
Módulo	4 — Pagos
Actor principal	Mercado Pago (webhook). También lo dispara el frontend cuando el cliente vuelve de pagar.
Actores secundarios	Cliente (recibe el resultado)
Descripción	El sistema consulta el estado real de un pago y actualiza la reserva: la confirma, la deja pendiente o devuelve el dinero.
Precondiciones	Existe un pago en Mercado Pago cuyo `external_reference` es el id de un pedido.
Disparador	`POST /api/pagos/notificacion` con `{ type: "payment", data: { id } }` (o los mismos datos en la URL).
Relaciones	Extendido por CU-16 Devolver pago `[pago no esperado]`
Endpoint	`POST /api/pagos/notificacion` → `200 { pedidoId, estadoPedido, estadoPago }` (público: no lleva token)

Flujo principal

1. Mercado Pago (o el frontend, al volver el cliente) envía el aviso con el id del pago.
2. La API ignora los avisos que no son de tipo `payment` (responde `200 { ignorado: true }`).
3. La API valida que el id sea numérico.
4. La API consulta `GET /v1/payments/{id}` a Mercado Pago: estado, monto, moneda y `external_reference`.
5. La API traduce el estado (`approved` → `APROBADO`, `rejected` → `RECHAZADO`, `cancelled` → `CANCELADO`, `refunded` / `charged_back` → `DEVUELTO`, el resto → `PENDIENTE`).
6. En una transacción, bloquea la fila del pedido (`FOR UPDATE`) para procesar de a un aviso.
7. Si el aviso ya se procesó (mismo id y mismo estado), no cambia nada (idempotente).
8. Si el pago está `APROBADO`, el pedido está `PENDIENTE_PAGO` y el monto y la moneda (ARS) coinciden: el pago pasa a `APROBADO` con el id de Mercado Pago y el pedido a `CONFIRMADO`; se registra `pago_aprobado`.
9. La API responde el estado del pedido y del pago; el frontend muestra “¡Pago aprobado! Tu reserva está confirmada.” y abre “Mis pedidos”.

Flujos alternativos

- **2a. Aviso de otro tipo (por ejemplo `merchant_order`):** `200 { ignorado: true }`, para que Mercado Pago no lo reintente.
- **3a. Id inválido:** `400`.
- **4a. El pago no existe en Mercado Pago:** `404`. **4b. Mercado Pago no responde:** `502` (Mercado Pago reintenta el webhook más tarde).
- **6a. El pago no corresponde a ninguna reserva:** `404 "El pago no corresponde a ninguna reserva"`.
- **8a. Pago pendiente, rechazado o cancelado:** el pago guarda ese estado y el pedido **sigue** `PENDIENTE_PAGO`; el cliente puede reintentar hasta que venza. El frontend muestra “El pago no se aprobó…” o “Mercado Pago está procesando el pago…”.

- **8b. Pago aprobado pero la reserva ya estaba cancelada o vencida, el monto no coincide o la reserva ya estaba pagada con otro pago:** se ejecuta CU-16 Devolver pago y se registra `pago_devuelto`. Si el pedido no estaba confirmado, el pago queda `DEVUELTO`.
- **8c. Devolución o contracargo hecho desde Mercado Pago sobre una reserva confirmada:** el pago pasa a `DEVUELTO` y el pedido a `CANCELADO` (libera el cupo).

Postcondiciones: el estado de la reserva coincide con el estado real del pago en Mercado Pago. Ninguna reserva se confirma sin un pago aprobado por el monto exacto.

Reglas: Mercado Pago es la única fuente de verdad del pago; repetir el aviso no cambia nada; dos avisos simultáneos del mismo pago se procesan de a uno.

CU-16 Devolver pago

Campo	Detalle
Módulo	4 — Pagos
Actor principal	(extiende CU-11 y CU-15)
Actores secundarios	Mercado Pago
Condiciones de extensión	CU-11: el pedido cancelado estaba pagado · CU-15: llegó un pago aprobado que no se esperaba
Endpoint externo	`POST https://api.mercadopago.com/v1/payments/{id}/refunds`

Flujo principal

1. La API pide a Mercado Pago la devolución total del pago, con la clave de idempotencia `devolucion-{id}` (si se repite, Mercado Pago no devuelve dos veces).
2. Mercado Pago confirma la devolución.
3. La API marca el pago como `DEVUELTO` y registra `pago_devuelto` en el log.

Flujos alternativos

- **2a. Mercado Pago falla:** `502`. En CU-11 la cancelación se deshace (el pedido sigue vigente); en CU-15 el aviso falla y Mercado Pago lo vuelve a enviar.

Postcondiciones: el cliente recupera el dinero.

CU-17 Vencer reservas impagas

Campo	Detalle
Módulo	4 — Pagos
Actor principal	(incluido en CU-05, CU-09, CU-10 y CU-14)
Descripción	Cancela las reservas cuyo plazo para pagar ya pasó, para que liberen el cupo.
Función	`vencerReservasImpagas()` en `lib/pagos.ts`

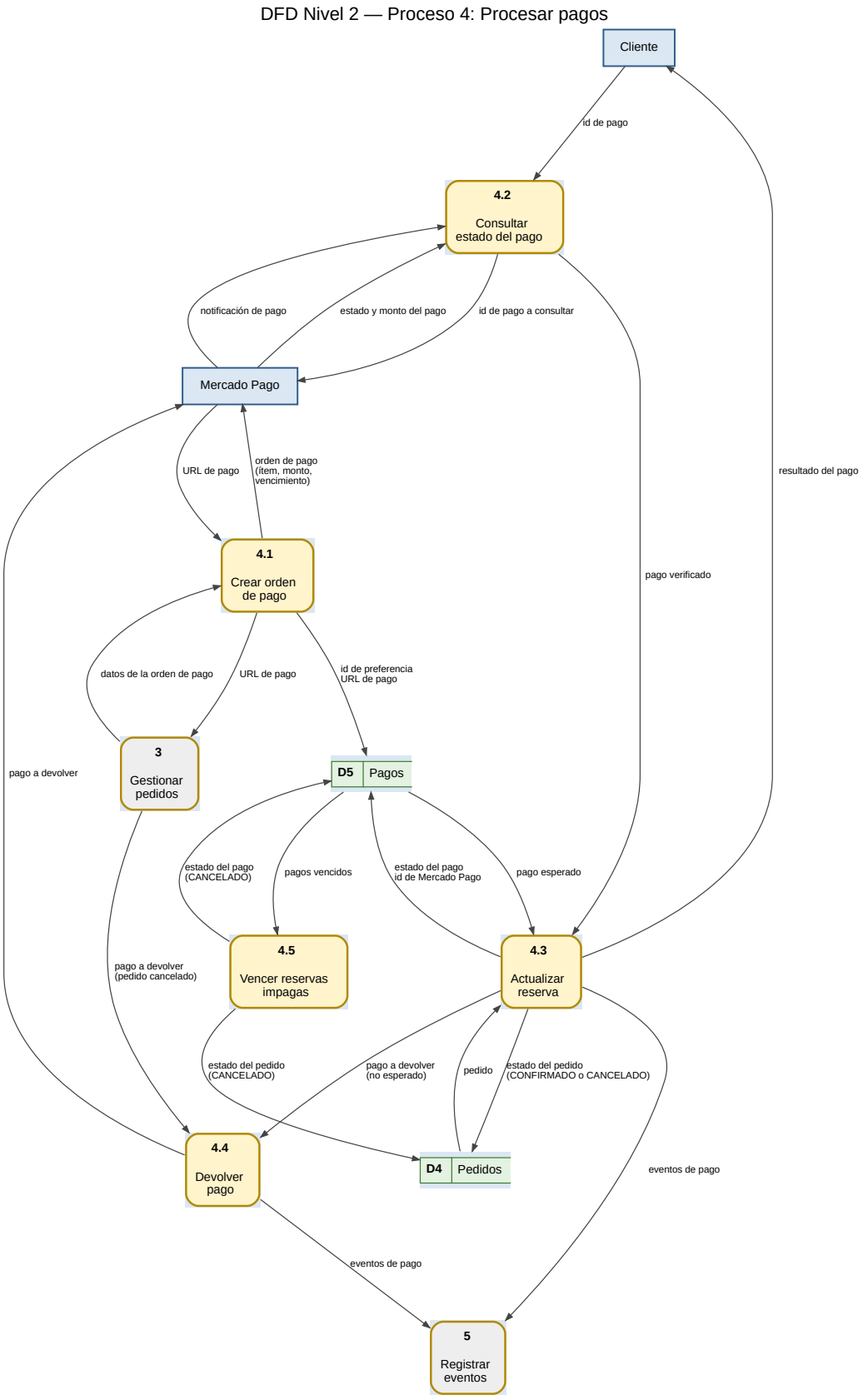
Flujo principal

1. La API busca los pagos `PENDIENTE` con `venceEn` vencido cuyo pedido sigue `PENDIENTE_PAGO` y los pasa a `CANCELADO`.
2. Pasa a `CANCELADO` los pedidos `PENDIENTE_PAGO` cuyo pago venció.

Postcondiciones: ninguna reserva vencida ocupa cupo.

Decisión de diseño: no hay una tarea programada (cron). El vencimiento se aplica “a demanda” justo antes de leer o contar el cupo, que es el único momento en que importa. Así no hace falta un proceso extra en el servidor (que además se duerme en el plan gratuito de Render). Como la preferencia de Mercado Pago vence a la misma hora, tampoco llegan pagos de reservas vencidas; si llegara uno, se devuelve (CU-15, 8b).

4. DFD Nivel 2 — Proceso 4 “Procesar pagos”



DFD Nivel 2 del proceso 4

N.º	Proceso	Entrada	Salida	Almacenes
4.1	Crear orden de pago	datos de la orden de pago (de 3)	orden de pago → Mercado Pago; URL de pago → 3	D5 (preferencia y URL)
4.2	Consultar estado del pago	notificación de pago (Mercado Pago), id de pago (Cliente)	id de pago a consultar → Mercado Pago; pago verificado → 4.3	—
4.3	Actualizar reserva	pago verificado	resultado del pago → Cliente; pago a devolver → 4.4; eventos → 5	D4, D5 (lee y escribe)
4.4	Devolver pago	pago a devolver (de 3 o 4.3)	pago a devolver → Mercado Pago; eventos → 5	—
4.5	Vencer reservas impagas	pagos vencidos	—	D5, D4 (escribe `CANCELADO`)

5. Datos que usa

Tabla	Campos	Notas
`Pago`	`pedidoId` (único), `monto`, `estado`, `preferenciaId` (único), `urlPago`, `mpPagoId`, `venceEn`, `actualizadoEn`	`monto` en pesos = cantidad × precio
`Pedido`	`estado`	Pasa a `CONFIRMADO` sólo desde `procesarPago`

6. API y servicios externos

Origen → destino	Método y ruta	Para qué
Mercado Pago → API	`POST /api/pagos/notificacion`	Webhook de pagos (en producción)
Frontend → API	`POST /api/pagos/notificacion`	Al volver de pagar (indispensable en local, donde el webhook no llega)
Frontend → API	`POST /api/pedidos/:id/pagar`	URL para reintentar el pago
API → Mercado Pago	`POST /checkout/preferences`	Crear la orden de pago
API → Mercado Pago	`GET /v1/payments/{id}`	Estado real del pago
API → Mercado Pago	`POST /v1/payments/{id}/refunds`	Devolución total

El webhook es público porque Mercado Pago no tiene token del sistema; no es un riesgo porque sólo recibe un **id** y el estado se consulta a Mercado Pago con la credencial privada.

7. Reglas y validaciones

Regla	Dónde
30 minutos para pagar, nunca después de la hora de corte	`vencimientoDelPago` en `domain/reglasPedido.ts`

Regla	Dónde
Sólo se confirma con pago aprobado, monto exacto y moneda ARS	`procesarPago` en `lib/pagos.ts`
Pago inesperado → devolución automática	`procesarPago` en `lib/pagos.ts`
Aviso repetido → sin cambios; avisos simultáneos → de a uno	`procesarPago` (comparación + `FOR UPDATE`)
Sin efectivo ni cajero (se acreditan tarde)	`crearPreferencia` en `lib/mercadoPago.ts`
El detalle de los errores de Mercado Pago no llega al usuario	`llamar()` en `lib/mercadoPago.ts`

Comportamiento ante falla o demora de Mercado Pago (consigna 3.5): sin credencial → `503`; sin conexión o con error → `502` con un mensaje claro, el detalle en el log y, al reservar, la reserva se cancela para no ocupar cupo. Si la falla es al confirmar, Mercado Pago reintenta el webhook y el cliente también puede volver a entrar a "Mis pedidos".

8. Pantallas

Momento	Qué ve el cliente
Después de "Reservar y pagar"	"Te llevamos a Mercado Pago para pagar tu reserva…" y la página de Mercado Pago
Al volver	"¡Pago aprobado! Tu reserva está confirmada.", "Mercado Pago está procesando el pago…", "El pago no se aprobó…" o "La reserva ya estaba cancelada o vencida: te devolvimos el pago."
"Mis pedidos"	"Pagado", "Falta pagar", "Pago rechazado", "Pago no completado", "Cancelado · pago devuelto"; botón "Pagar $…"

9. Tests que lo cubren (`pagos.test.ts`, 14)

Reserva pendiente con pago del 100 % y URL; confirmación sólo con pago aprobado; rechazado, cancelado o pendiente no confirman y se puede reintentar; aviso con datos en la URL y avisos de otro tipo ignorados; pago inexistente (404) e id inválido (400); monto distinto → devolución; pago de una reserva cancelada → devolución; cancelar una reserva pagada → devolución; devolución desde Mercado Pago → cancela; reserva sin pagar que vence y libera el cupo; no se paga un pedido ajeno ni uno ya pagado; falla de Mercado Pago al crear el pago → reserva cancelada; menú sin precio. Mercado Pago se simula reemplazando `fetch` (`simularMercadoPago()` en `tests/integration/ayudantes.ts`): los tests nunca llaman al servicio real.

10. Para modificarlo

Si piden…	Tocar
Cambiar el plazo para pagar	`MINUTOS_PARA_PAGAR` en `domain/reglasPedido.ts` (y su test)
Agregar un tiempo máximo de espera a Mercado Pago	`fetch` en `llamar()` de `lib/mercadoPago.ts` (por ejemplo `signal: AbortSignal.timeout(10000)`) y manejar el error como `502`

Si piden…	Tocar
Mostrar el id de pago de Mercado Pago al cliente	`select` del `pago` en `GET /api/pedidos` (`routes/pedidos.ts`) → `tipos.ts` → `Pedidos.tsx`

Módulo 5 — Plataforma (transversal)

1. Propósito

Reúne lo que usan todos los módulos: cómo se arma la API, cómo se validan los datos y se responden los errores, cómo se registran los eventos (observabilidad), el chequeo de salud, la configuración por variables de entorno, los tests y el pipeline de CI/CD. El registro de eventos es el **proceso 5 "Registrar eventos"** de los DFD. No tiene casos de uso propios: da soporte a los CU-01 a CU-17.

2. Dónde está en el código

Archivo	Qué hace
`backend/src/server.ts`	Punto de entrada: lee la configuración y levanta el servidor
`backend/src/app.ts`	Arma la app Express: `helmet`, CORS, JSON de hasta 100 kB, logs HTTP, rutas y manejo de errores
`backend/src/config/env.ts`	**Única** lectura de variables de entorno; falla al arrancar si falta `DATABASE_URL` o `JWT_SECRET`
`backend/src/lib/errors.ts`	`HttpError(status, mensaje, codigo?)`
`backend/src/middleware/errorHandler.ts`	`noEncontrado` (404) y `manejadorErrores` (400 / 4xx / 500)
`backend/src/lib/logger.ts`	Logger `pino` en JSON, con datos sensibles ocultos
`backend/src/routes/health.ts`	`GET /api/health`
`backend/src/lib/prisma.ts`	Cliente único de Prisma
`frontend/src/api/cliente.ts`	Única puerta del frontend a la API (`pedirApi`, `enviar`, manejo de 401)
`backend/vitest.config.ts`	Tests, base de pruebas y umbral de cobertura del 65 %
`.github/workflows/ci-cd.yml`	Pipeline de CI/CD
`docker-compose.yml`	Bases locales de desarrollo (5432) y de tests (5433)

3. Validación y errores

Toda entrada se valida con **zod** al principio de cada ruta (`esquema.parse(req.body)`). Si algo falla, el error viaja solo hasta el manejador central (Express 5 lo hace también en rutas `async`):

Situación	Respuesta
Datos que no cumplen el esquema (`ZodError`)	`400 { error: "Datos inválidos", detalles: [{ campo, mensaje }] }`
JSON mal formado	`400 { error: "El cuerpo de la petición no es un JSON válido" }`
Regla de negocio, permiso o recurso (`HttpError`)	su código (401, 403, 404, 409, 502, 503) con `{ error, codigo? }`
Ruta inexistente	`404 { error: "Recurso no encontrado", ruta }`

Situación	Respuesta
Cualquier otro error	`500 { error: "Error interno del servidor" }`, sin detalles internos; el detalle va al log

Ningún dato inválido llega a la base ni termina en un 500.

4. Observabilidad (proceso 5 “Registrar eventos”)

La API escribe **logs estructurados en JSON** (pino) en la salida estándar; en producción los guarda y muestra **Render Logs**. Cada línea tiene la hora, el nivel, `servicio: "viandassemana-api"` y, en los eventos de negocio, un campo `evento` por el que se puede buscar. `pino-http` agrega una línea por petición con método, ruta, código de respuesta y tiempo.

`evento`	Nivel	Cuándo	Origen
`login_fallido`	warn	Email o contraseña incorrectos	`routes/auth.ts`
`recuperacion_solicitada`	info	Se mandó un código de recuperación	`routes/auth.ts`
`password_restablecida`	info	Se cambió una contraseña con código	`routes/auth.ts`
`regla_negocio`	warn	Cualquier rechazo con código: `CUPO_COMPLETO`, `FUERA_DE_HORARIO`, `YA_CANCELADO`, `SIN_PRECIO`, `NO_PAGABLE`, `CODIGO_*`	`middleware/errorHandler.ts`
`pago_aprobado`	info	Reserva pagada y confirmada	`lib/pagos.ts`
`pago_devuelto`	info / warn	Devolución por cancelación o por pago no esperado	`routes/pedidos.ts`, `lib/pagos.ts`
`mercadopago_sin_configurar` / `_sin_conexion` / `_error`	error	Falla de Mercado Pago	`lib/mercadoPago.ts`
`correo_sin_configurar` / `correo_fallido`	error	Falla del correo	`lib/mailer.ts`
`error_servidor`	error	Error no controlado (500)	`middleware/errorHandler.ts`

Para qué sirve: si un cliente dice “no pude pedir”, se busca en Render Logs a esa hora el evento `regla_negocio` y se ve si fue por `CUPO_COMPLETO`, por `FUERA_DE_HORARIO` o por un `error_servidor`.

Qué nunca se registra: el encabezado `Authorization`, las cookies, las contraseñas ni los hashes (`redact` en `lib/logger.ts`).

5. Salud y seguridad de la API

- `GET /api/health` → `200 { estado: "ok", servicio, hora }`. Lo usa Render para saber si la API está viva y sirve para “despertarla”.
- `helmet` agrega encabezados de seguridad HTTP.

- CORS sólo acepta los orígenes de `CORS_ORIGIN` (en producción, la URL del frontend) y los métodos y encabezados que usa la app.
- El cuerpo JSON tiene un límite de 100 kB.
- Ningún secreto está en el código: todos salen de `config/env.ts`.

6. Configuración

Variable	Obligatoria	Uso
`DATABASE_URL`	sí	Conexión a PostgreSQL
`JWT_SECRET`	sí	Firma de tokens y HMAC de códigos de recuperación
`PORT`, `NODE_ENV`, `CORS_ORIGIN`, `LOG_LEVEL`	no	Puerto, entorno, orígenes permitidos, nivel de log
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	no (sin ellas, recuperar contraseña da 503)	Correo
`MP_ACCESS_TOKEN`, `FRONTEND_URL`, `API_URL`	no (sin token, reservar da 503)	Mercado Pago
`VITE_API_URL` (frontend)	no (por defecto `http://localhost:3000`)	URL de la API, se fija al construir

7. Tests

`npm test` (en `backend/`) corre **78 tests**: 20 unitarios y 58 de integración, y genera la cobertura.

Archivo	Tipo	Tests
`tests/unit/reglasPedido.test.ts`	unitario	19
`tests/unit/mailer.test.ts`	unitario	1
`tests/integration/auth.test.ts`	integración	11
`tests/integration/recuperacion.test.ts`	integración	7
`tests/integration/menus.test.ts`	integración	10
`tests/integration/pedidos.test.ts`	integración	11
`tests/integration/pagos.test.ts`	integración	14
`tests/integration/health.test.ts`	integración	5 (salud, CORS, 404 y JSON mal formado)

- Los de integración hacen la petición HTTP completa con `supertest` contra una **base de pruebas separada** (`localhost:5433/viandas_test`), nunca la de desarrollo ni la de producción. Cada archivo limpia sus tablas y los archivos corren de a uno.
- Mercado Pago se simula reemplazando `fetch`, y el correo con `vi.mock`.
- **Cobertura:** si las líneas cubiertas bajan del **65 %**, `npm test` falla. Sólo se excluyen `src/server.ts` (punto de entrada) y `src/config/**` (lectura de variables). El informe queda en `backend/coverage/index.html`.

8. Pipeline de CI/CD

Se dispara con cada `push` y cada pull request a `main` (rama protegida: los cambios entran por PR).

Job	Pasos	Cuándo corre
`backend`	`npm ci` → `prisma generate` → `prisma migrate deploy` sobre una base PostgreSQL efímera → `npm test` (con umbral 65 %) → sube la cobertura como artefacto → `npm run build`	push y PR
`frontend`	`npm ci` → `npm run build`	push y PR
`desplegar`	`curl` a los *deploy hooks* de Render del backend y del frontend	sólo en push a `main` y sólo si `backend` y `frontend` pasaron

Si falla un test o baja la cobertura, `desplegar` no corre y producción no cambia. El recorrido completo, con los secretos, está en el documento de arquitectura y despliegue.

9. Para modificarlo

Si piden…	Tocar
Un manejo de error que no existe	lanzar `new HttpError(status, 'mensaje', 'CODIGO')` donde ocurre; el manejador central ya lo responde y lo registra
Un evento nuevo en los logs	`logger.info({ evento: 'nombre', ...datos }, 'mensaje')`
Una variable de entorno nueva	`config/env.ts`, `.env.example`, README y el panel de Render